

# Shared training catalogue

exp022 · 28 June 2026 · Draft

## Abstract

This entry is the gamma-gated-sparsity collection’s training catalogue. It defines each type of training, records the resulting learning curves and activity diagnostics, and exposes one shared checkpoint bank to downstream experiments. The completed bank contains 87 independently trained cells across the canonical, spike-budget, inhibitory-timescale, timestep, and recurrent-initialization studies. A sixth training type is planned for streaming inference: PING trained with a variable Poisson input rate and a summed-spiking readout. That extension is specified here but is not counted among the completed cells.

## Methods

### 1. Shared architecture and training contract

1. **Encode the stimulus.** Each MNIST image supplies 784 independent Poisson input channels. Completed training types use a fixed maximum-pixel rate of 25 Hz. The planned streaming bank instead samples one rate independently for each image presentation.
2. **Simulate the network.** A trial lasts 200 ms. The standard timestep is 0.1 ms, giving 2000 recurrent updates per presentation. PING contains 1024 excitatory and 256 inhibitory neurons; COBA retains the excitatory pathway but disables recurrent E/I coupling.
3. **Form the readout.** Completed cells use **mem-mean**: a 1024-to-10 projection drives ten output LIF membranes and the class logits are their time-averaged voltages. The planned variable-rate cells use **rate**: the 1024 excitatory spike trains are summed through time and projected once through a 1024-to-10 linear readout.
4. **Train and retain.** Every registered configuration trains seeds 42, 43, and 44 for 50 epochs. The canonical reference sees 70000 pooled MNIST samples; sweep cells see 7000. Each cell retains its configuration, weights, epoch metrics, and a representative held-out raster.

| Component                        | Shape                   | Role                                 |
|----------------------------------|-------------------------|--------------------------------------|
| Poisson input                    | $T \times B \times 784$ | one event stream per image pixel     |
| input projection $W_{\text{in}}$ | $784 \times 1024$       | pixels to excitatory population      |
| excitatory population            | $B \times 1024$         | trained stimulus representation      |
| inhibitory population            | $B \times 256$          | PING feedback population             |
| E→I projection $W_{\text{EI}}$   | $1024 \times 256$       | excitatory recruitment of inhibition |
| I→E projection $W_{\text{IE}}$   | $256 \times 1024$       | inhibitory feedback                  |
| class readout $W_{\text{out}}$   | $1024 \times 10$        | excitatory activity to digit logits  |

Here  $T$  is the number of timesteps and  $B$  is the minibatch size. Matrix shapes use the stored source-to-destination orientation.

## 2. Registered training types

The registry is organised by scientific training type. A type may contain several parameter variants, and every variant contains three independent seed cells.

| Training type                     | Models     | Varied parameter                                      | MNIST         | Cells | Trained | Primary use                        |
|-----------------------------------|------------|-------------------------------------------------------|---------------|-------|---------|------------------------------------|
| full-data reference               | COBA, PING | none                                                  | all           | 6     | 6       | canonical accuracy                 |
| spike-budget training             | COBA, PING | $\theta_u \in \{\text{off}, 5, 2, 1, 0.5, 0.2\}$      | 10%           | 36    | 36      | accuracy–sparsity frontier         |
| inhibitory-timescale training     | PING       | $\tau_{\text{GABA}} \in \{4.5, 6, 9, 12, 18, 27\}$ ms | 10%           | 18    | 18      | rhythm timescale                   |
| timestep training                 | PING       | $\Delta t \in \{0.05, 0.1, 0.25, 0.5, 1\}$ ms         | 10%           | 15    | 15      | numerical stability                |
| recurrent-initialization training | PING       | loop initialization and trainability                  | 10%           | 12    | 12      | built-in versus learned recurrence |
| variable-rate streaming training  | PING       | input-rate distribution and readout                   | 10% initially | 3     | 0       | exp082 streaming inference         |

**Note, planned variable-rate bank.** Exp080 selects the interval 0.5–25 Hz for later PING training. The proposed cells sample uniformly from the discrete set 0.5, 1, 2, 5, 10, and 25 Hz, independently per presentation, and use the summed-spiking **rate** readout. This training changes both the input distribution and readout relative to the existing `ping_off_seed*` cells. It is therefore a new training type, not another point in the spike-budget sweep.

**Note, engine support.** `tools/snn` accepts the categorical rate set through `--input-rates`, samples it reproducibly per presentation, serializes both the values and sampling rule, and restores them through `--load-config`. The three cells are registered for the Cambridge job array. This entry still does not report their results because the full training jobs have not run.

## 3. Cambridge HPC compute plan

Surrogate-gradient backpropagation stores state across every timestep. A standard presentation therefore traverses 2000 sequential recurrent steps, and the backward pass is both memory-heavy and bandwidth-sensitive. The completed registry represents approximately 185 A100-GPU-hours and 49 million sample-forwards. The  $\Delta t = 0.05$  ms cells are the exceptional memory case, previously requiring approximately 31 GB.

The Cambridge run should use one independently resumable job-array element per cell. Each element writes only its own cell directory, validates the emitted configuration before treating an existing checkpoint as complete, and records the scheduler job identifier with the run provenance. Canonical full-data cells should not share an allocation with sweep cells because their runtime is substantially longer. The final array layout, GPU type, memory request, concurrency cap, and wall-time request remain notes until the available Cambridge partition is confirmed.

## Results

Each completed training type is reported in the same order: definition, learning curve, representative seed-42 raster, and outcome. The complete raster gallery remains in the appendix.

### 1. Full-data reference

coba and ping with no spike budget, all 70k images, seeds 42/43/44 (6 cells), the full-data baseline. Unconstrained, the two architectures are essentially tied: coba reaches  $\approx 95.5\%$  and ping  $\approx 94.0\%$ , with near-zero across-seed spread. This is the reference point from which the spike-budget sweep pulls them apart.

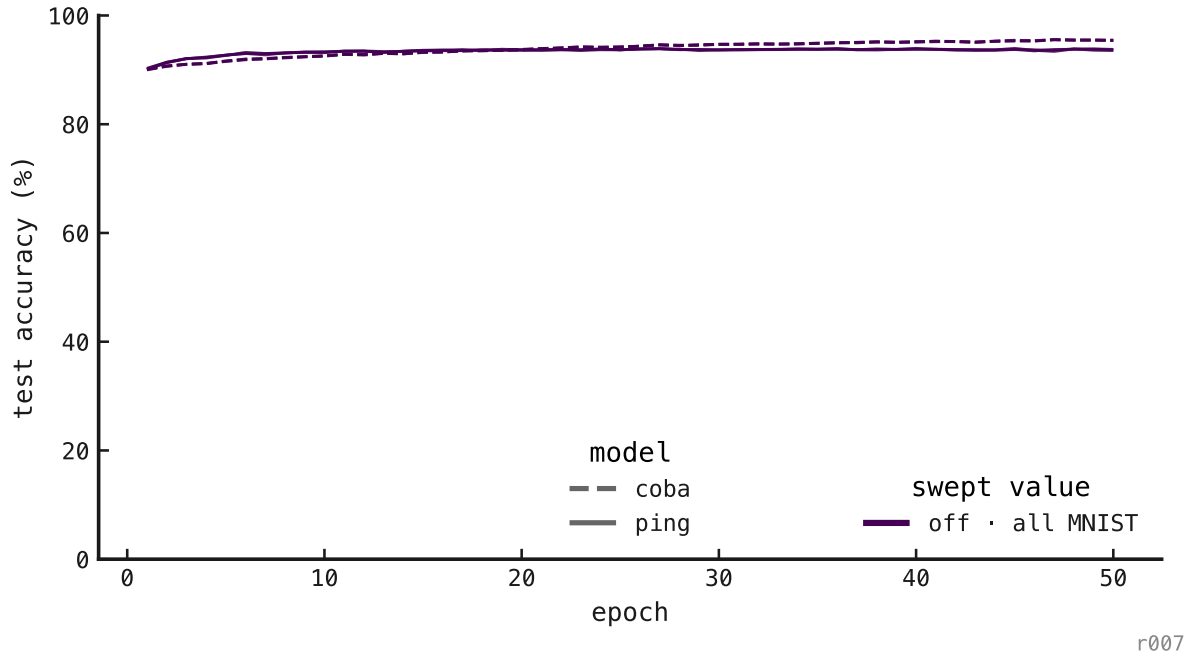


Figure 1: Test accuracy over epochs, canonical full-MNIST cells (coba dashed, ping solid; three seeds each). The two loops reach parity when no spike budget is imposed.

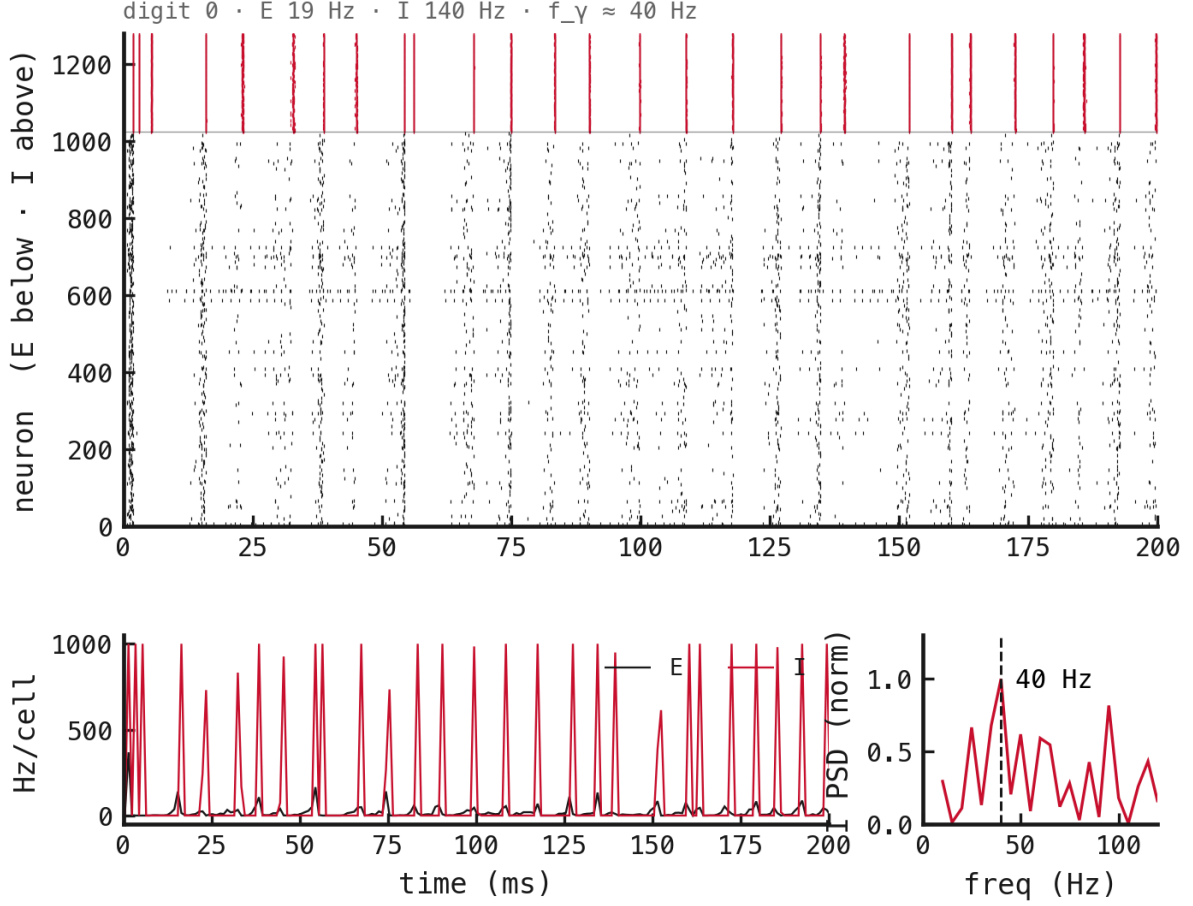


Figure 2: Representative activity after full-data PING training. The raster shows excitatory and inhibitory spikes for the common held-out digit-0 probe; the lower panel shows the population-rate diagnostic.

## 2. Spike-budget training

coba and ping across spike budgets  $\in$  off, 5, 2, 1, 0.5, 0.2, three seeds each (36 cells), so the accuracy–rate frontier carries error bars at every point. The spike budget is a per-neuron cap on firing (spikes/trial); lower is tighter. This is the headline result: as the budget tightens, ping degrades gracefully ( $91.6 \rightarrow 86.5$  %) while coba collapses ( $90.7 \rightarrow 60.1$  %), a gap that widens monotonically from  $\approx 0$  to  $\approx 26$  points. ping’s  $\gamma$ -rhythm gates sparsity; coba’s bare feedforward code cannot pay the budget without shedding accuracy.

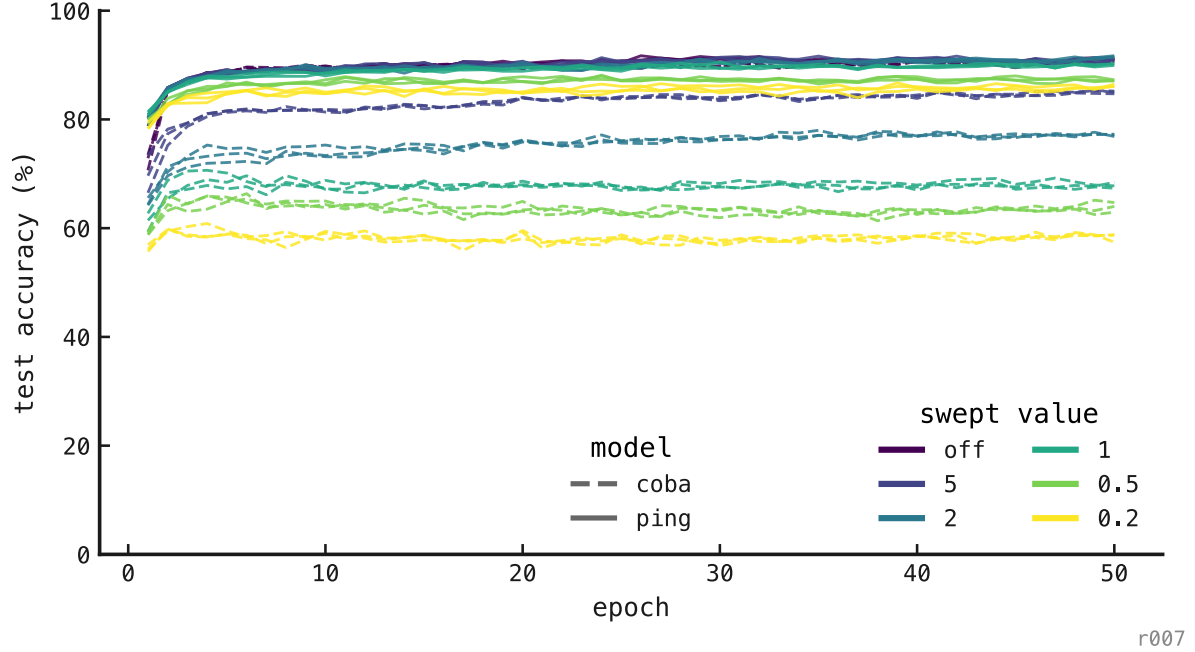


Figure 3: Test accuracy over epochs across the spike-budget sweep. Tighter budgets plateau lower (the spike-economy trade-off), and coba falls far faster than ping.

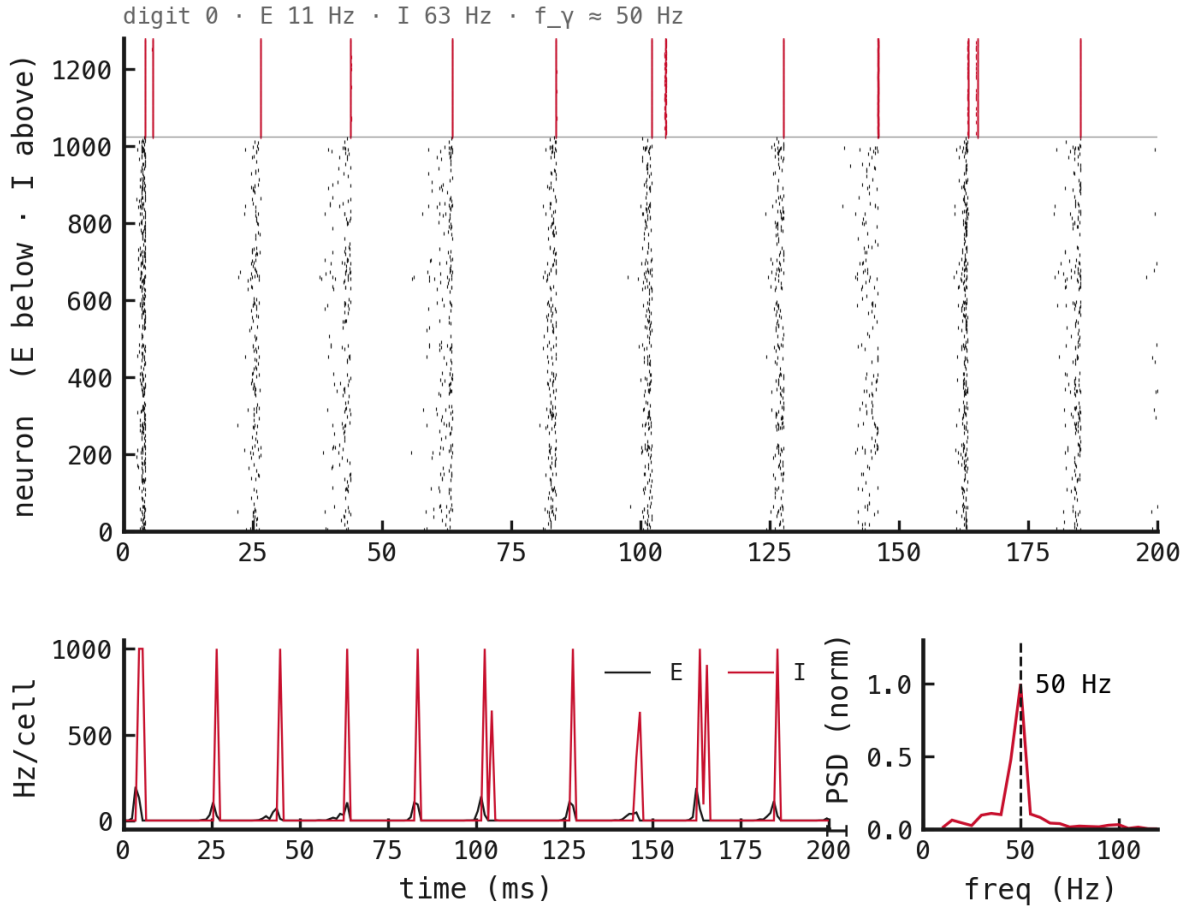
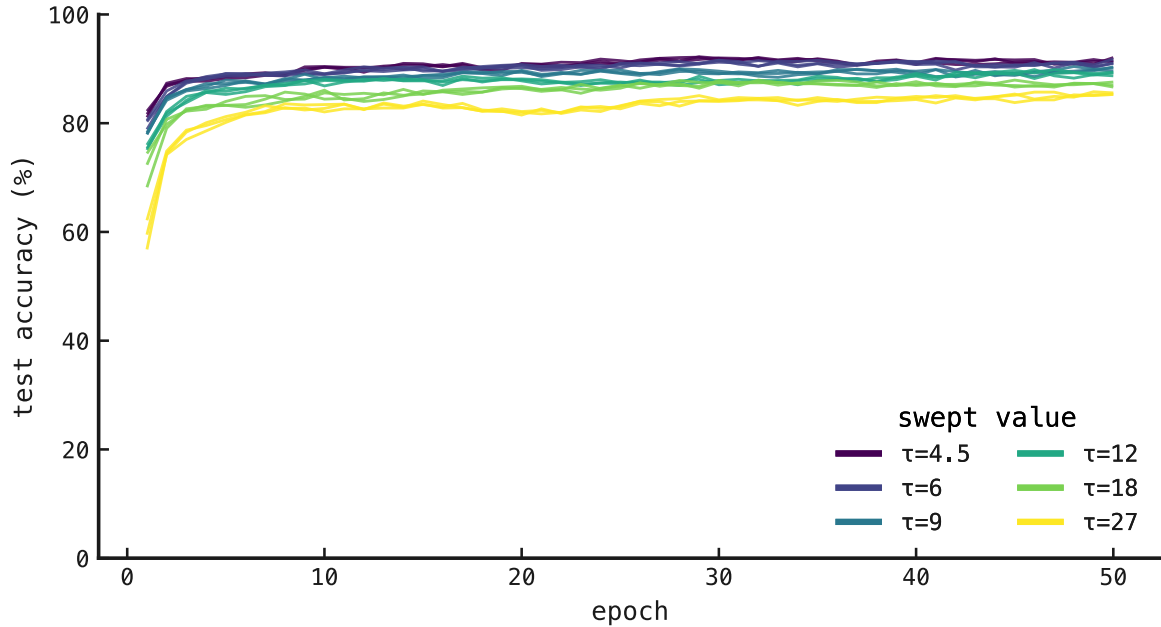


Figure 4: Representative activity for the spike-budget training type at the no-budget endpoint. The raster uses seed 42 and the common held-out digit-0 probe. The appendix shows every budget for both architectures.

**Scope.** This frontier is a 10%-MNIST result: the whole sweep trains on the subset (§2 of Methods), and we have not re-run it at full data. The comparison is nonetheless internally clean: every cell here shares the same data fraction, so the cobra-vs-ping gap is a budget effect, not a data-fraction artifact. And the direction of the density difference (§7) makes 10% the **conservative** regime for this claim: at full MNIST the no-budget baseline fires  $\approx 2\times$  harder, so a tight per-neuron cap would have further to pull cobra down, widening the gap rather than closing it. We therefore read the  $\approx 26$ -point separation as a floor, and do not claim the absolute numbers transfer unchanged to full MNIST.

### 3. Inhibitory-timescale training

ping across  $\tau_{\text{GABA}} \in 4.5, 6, 9, 12, 18, 27$  ms, three seeds each (18 cells). Accuracy is largely insensitive to inhibitory decay ( $\approx 88\text{--}92\%$  across the ladder), but the **rhythm** is not: measured from the trained networks, the  $\gamma$ -frequency falls monotonically with  $\tau_{\text{GABA}}$  ( $\approx 50$  Hz at 4.5 ms  $\rightarrow \approx 19$  Hz at 27 ms), sitting at  $\approx 45$  Hz at the canonical  $\tau_{\text{GABA}} = 6$  ms, matching the operating point (Appendix).



r007

Figure 5: Test accuracy over epochs across the  $\tau_{\text{GABA}}$  ladder; cells converge to similar accuracy regardless of inhibitory decay.

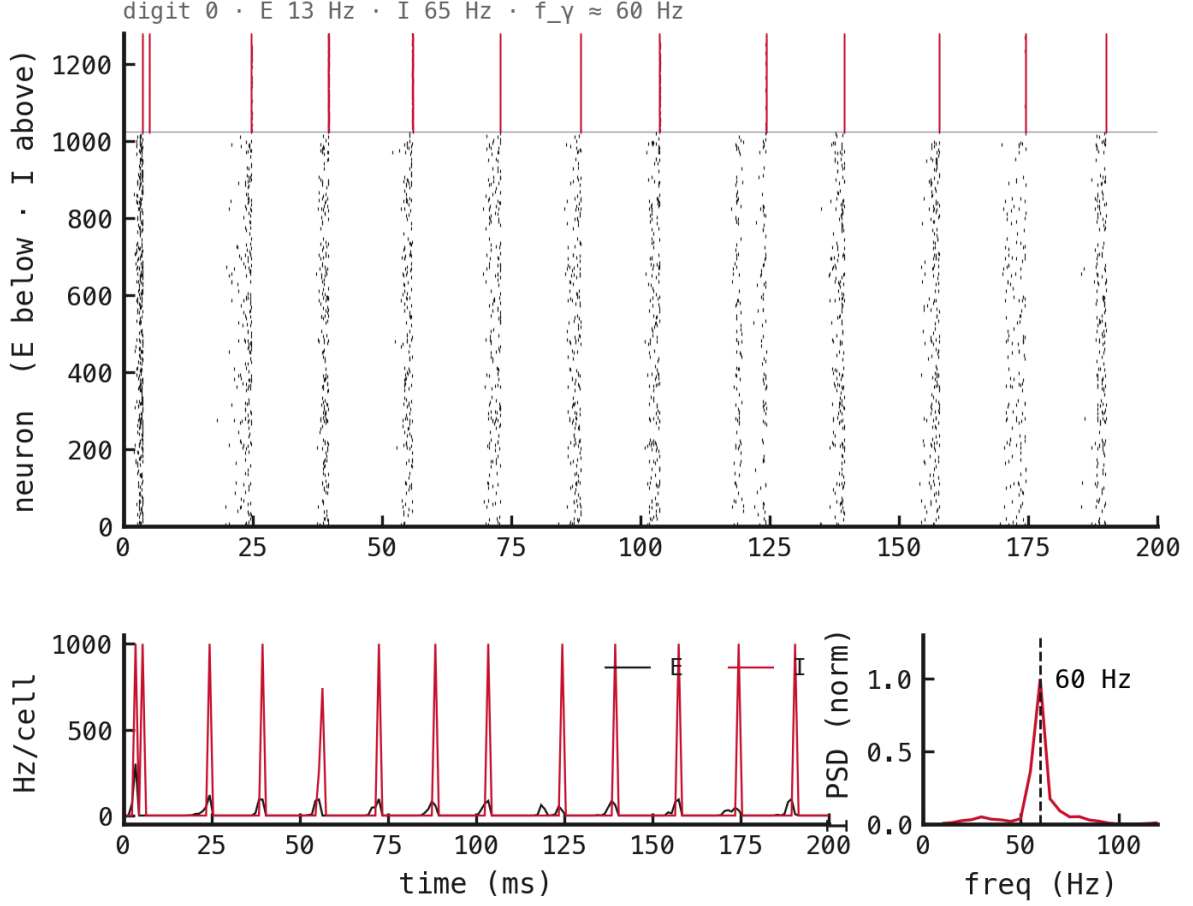
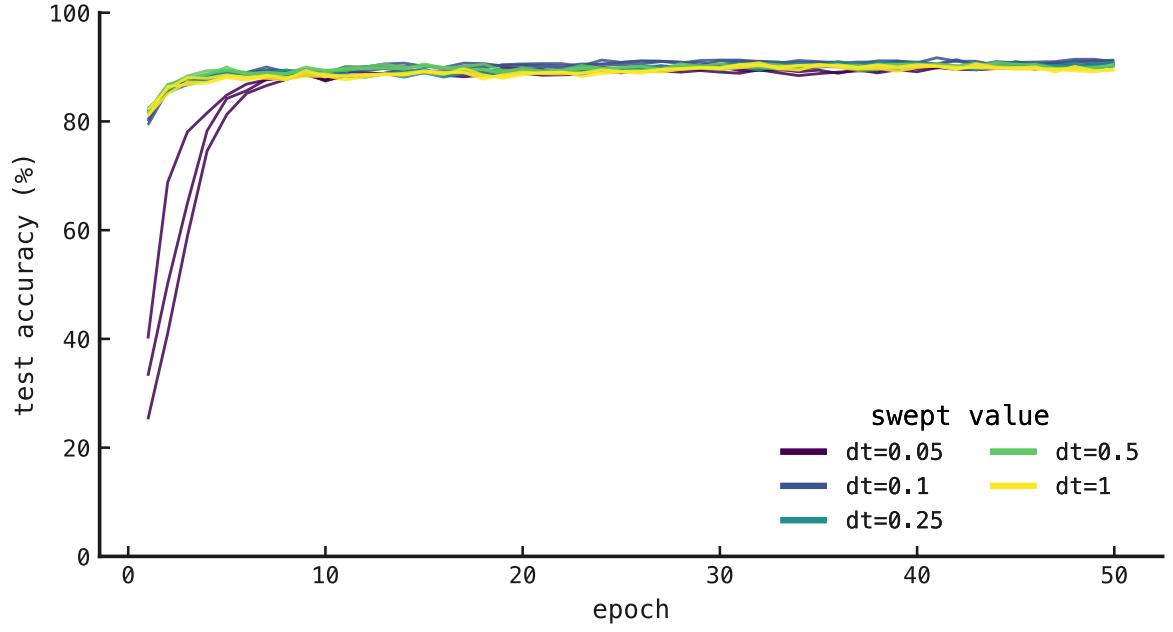


Figure 6: Representative activity for inhibitory-timescale training at the standard  $\tau_{\text{GABA}}$ . The raster uses seed 42 and the common held-out digit-0 probe; the appendix shows the complete ladder.

#### 4. Timestep training

ping across  $\Delta t \in 0.05, 0.1, 0.25, 0.5, 1.0$  ms (physical T fixed), three seeds each (15 cells), the documented timestep exception. Accuracy is flat across the sweep ( $\approx 90.4\text{--}91.4\%$ ): the integrator is robust to timestep from 0.1 to 1.0 ms, and the 0.05 ms cells (which need  $\approx 31$  GB and so ran on a 5090) agree.



r007

Figure 7: Test accuracy over epochs across the integration-timestep sweep; accuracy is insensitive to  $\Delta t$  over the tested range.

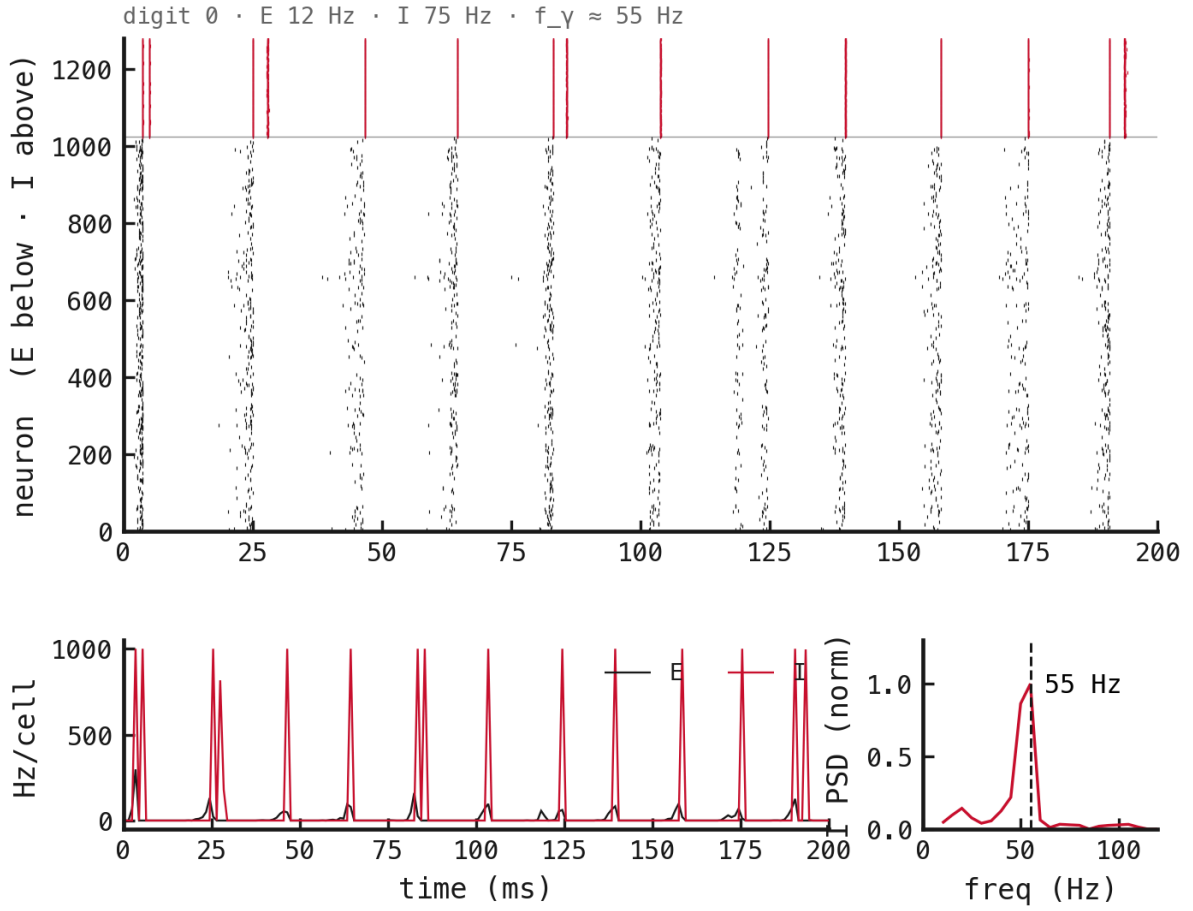


Figure 8: Representative activity for timestep training at the standard  $\Delta t$ . The raster uses seed 42 and the common held-out digit-0 probe; the appendix shows all timestep variants.

## 5. Recurrent-initialization training

ping with four recurrent-loop inits (frozen PING, trainable from PING / zero / small seed), three seeds each (12 cells). All reach  $\approx 89\text{--}91\%$ , but only the frozen-PING control keeps the true E/I regime ( $E \approx 10\text{ Hz}$ ,  $I \approx 62\text{ Hz}$ ): the trainable-loop cells drift toward a feedforward code (high E, low or zero I): the zero-init cells never engage inhibition at all ( $I \approx 0\text{ Hz}$ ). Comparable accuracy, but the rhythm is not learned when it is not built in.

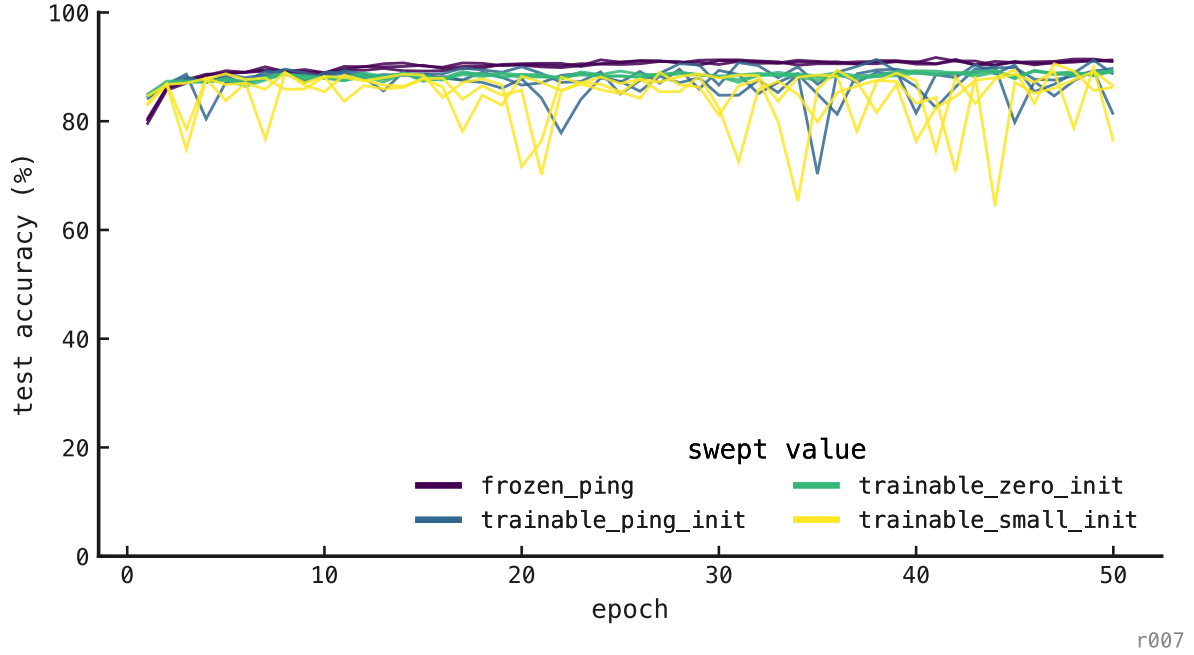


Figure 9: Test accuracy over epochs across the recurrent-loop inits; trainable-loop cells learn noisier curves than the frozen control.

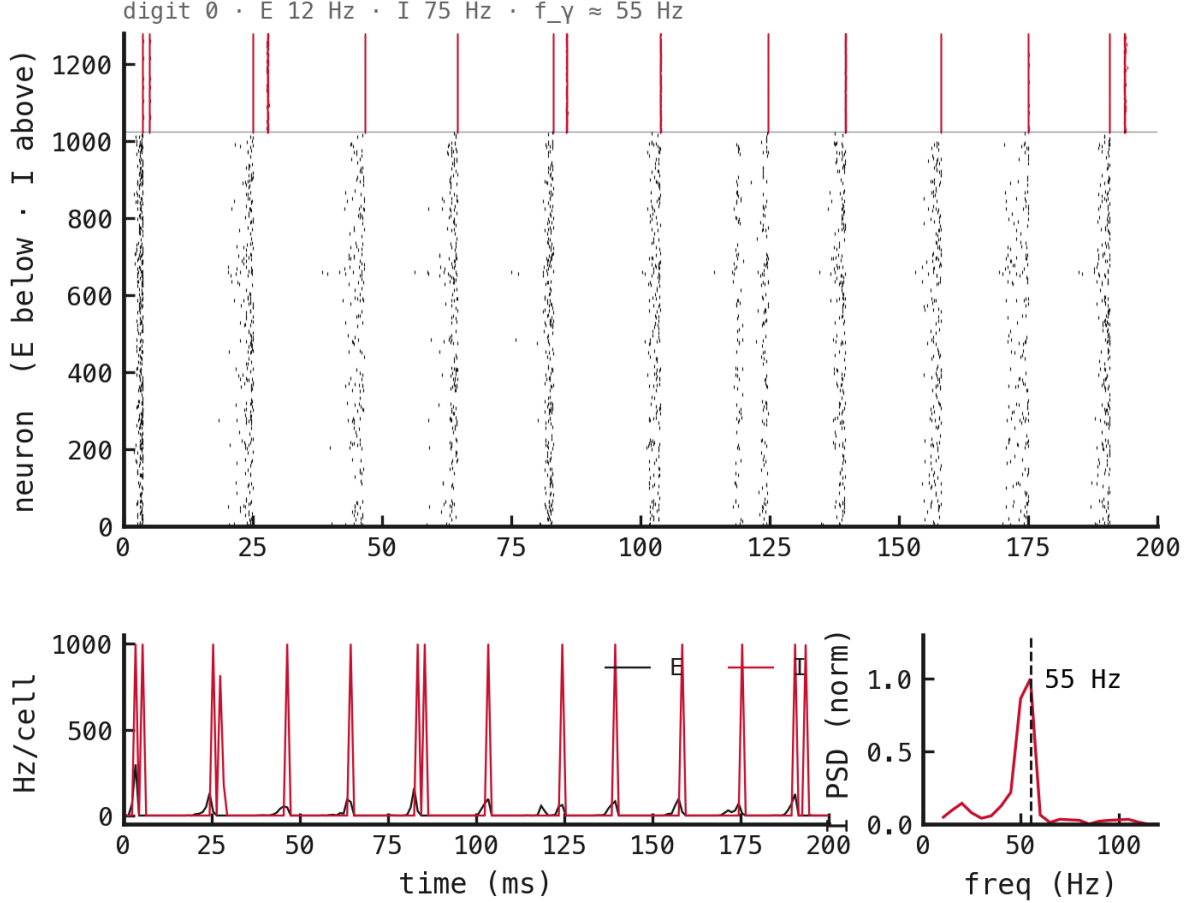


Figure 10: Representative activity for recurrent-initialization training. This seed-42 control keeps the recurrent PING loop frozen; the appendix shows all four initialization and trainability conditions.

## 6. Variable-rate streaming training

**Note, awaiting training.** No learning curve or raster exists yet. After the registered Cambridge jobs complete, this section will show the three mixed-rate learning curves, fixed-rate held-out accuracy across 0.5–25 Hz, and matched rasters at low, intermediate, and high rates. It will compare the new **rate**-readout cells with the existing fixed-25-Hz **mem-mean** cells. The checkpoints are the training source for exp082, which supersedes exp048 for variable-rate streaming inference.

## 7. Training data and spike density

The appendix rasters split cleanly along one axis that is easy to miss: the **canonical** cells see all 70k MNIST images, but every sweep (including the no-budget spike-budget = off cell, which is otherwise identical to canonical) trains on 10%. That difference deserves its own read, because it changes how **busy** the trained network is before any sweep parameter enters.

We isolate it by sending the **same** fixed digit-0 image through the no-budget cobra and ping networks at each fraction (seed 42). Same architecture, same operating point ( $\tau_{\text{AMPA}} = 2$  ms,  $\tau_{\text{GABA}} = 6$  ms), same spike budget (none): the only difference is  $10\times$  the training images, so any gap in the rasters is the data’s doing alone.

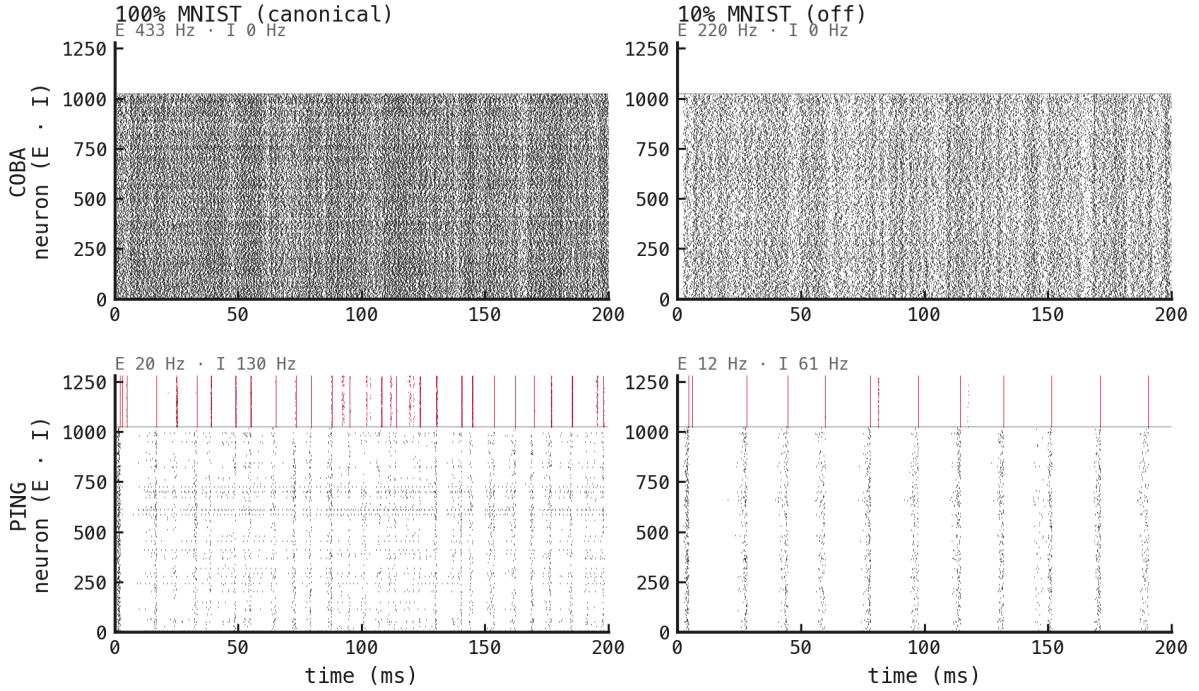


Figure 11: The same digit-0 image through the no-budget cobra (top) and ping (bottom) networks, trained on all of MNIST (left) versus 10% (right); E cells black below the divider, I cells red above, per-panel mean rates annotated. More training data yields a visibly denser code in both architectures.

More data buys a denser code. In both loops the full-MNIST network fires roughly twice as hard:

- **coba** excitatory rate  $\approx 420$  Hz on all of MNIST against  $\approx 212$  Hz on 10%;
- **ping** inhibitory rate  $\approx 140$  Hz against  $\approx 63$  Hz (excitatory  $\approx 19$  vs  $\approx 11$  Hz).

where the **mean rate** is a population’s total spikes divided by its neuron count and the 200 ms window. The canonical rasters are visibly busier: the extra spikes are the network recruiting capacity to separate the fuller set of class variations, which the 10% subset never forces it to. ping keeps its  $\gamma$  rhythm at both fractions, and the sparser 10% network even reads as a **cleaner** gamma (Appendix A.2); cobra stays asynchronous throughout (I silent, no loop). The density shift is a change of degree, not regime.

The practical consequence is a caveat on the appendix as a whole: **absolute** firing rates are not comparable across the canonical-vs-sweep boundary, because the 10% cells sit at a systematically lower density for reasons that have nothing to do with the swept parameter. This is exactly why the canonical reference (§1) carries the headline rates and the sweeps are read as **trends within a family** (the spike-budget frontier, the  $\tau_{\text{GABA-to-}\gamma}$  scaling) rather than as absolute numbers set against the full-data baseline.

## 8. Why more data drives a denser code

The figure measures the density gap; it does not prove its cause. But only two things change between a row’s two cells (the number of training images and, downstream of that, the number of weight updates), and both push firing **up**. Neither is specific to the E/I loop,

which is why cobra and ping move together rather than one architecture reacting and the other not.

- **More to separate.** The 70k-image set carries far more within-class variation than its 10% subset. To keep the ten digit classes separable at the readout (a linear map from per-neuron spike counts to class scores), the network must partition a higher-dimensional representation, and it pays for that resolution in spikes: more neurons recruited, each firing more. The smaller subset is an easier separation problem that a sparser code already fits, so the network never has to spend the extra spikes.
- **More weight updates.** Epochs are fixed at 50 for both, but an epoch over 70k images is  $\approx 10\times$  the mini-batches of an epoch over 10%, so the canonical cells take  $\approx 10\times$  as many gradient steps. These are the **no-budget** cells, so nothing opposes the drift: each update is free to grow the input and recurrent weights that set the drive  $g(V - E)$ , and more updates compound into a higher operating point.

where the terms are:

- $g$  — the synaptic conductance a presynaptic spike opens (larger weights  $\rightarrow$  larger  $g$ );
- $V$  — the postsynaptic membrane voltage;
- $E$  — the synapse’s reversal potential, so  $g(V - E)$  is the current a spike injects.

The spike budget is precisely the counter-pressure to both levers: the sweep’s tightening  $\theta_u$  (the per-neuron cap on spikes/trial) caps the rate growth described here, which is why the budgeted cells sit far below their no-budget siblings. The canonical-off pair simply removes that cap, leaving data fraction as the only lever. A clean way to separate the two mechanisms would be to train a 10% cell for  $\approx 10\times$  the epochs, matching the update count at the smaller set: if the rate closes most of the gap the growth is update-driven, and if it does not the residual is the genuine cost of the harder separation. We have not run that control; the two levers are offered as the mechanism the rasters are consistent with, not as a decomposition we have measured.

## Appendix — per-config spike rasters (digit 0)

The same fixed MNIST image (digit 0, sample 0) sent through each trained network (seed 42 as the per-config representative), so every raster is directly comparable. E cells sit below the divider, I cells above; the lower panel is the 1 ms population rate. These are the visual counterpart to the firing-rate and rhythm numbers above: sparse  $\gamma$ -rhythmic PING versus dense asynchronous COBA, and how each regime deforms under the sweeps.

## A.1 Canonical reference

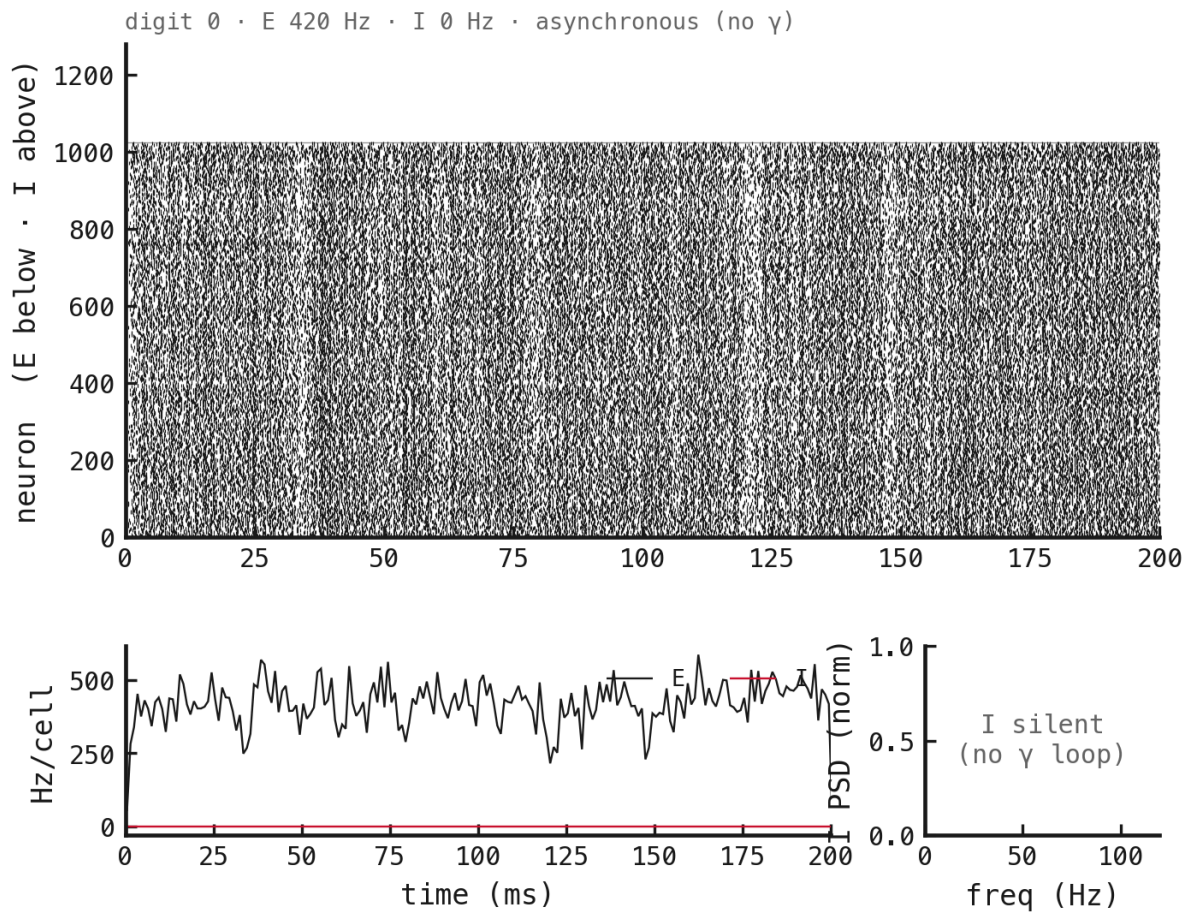


Figure 12: COBA, canonical (no spike budget, all MNIST).

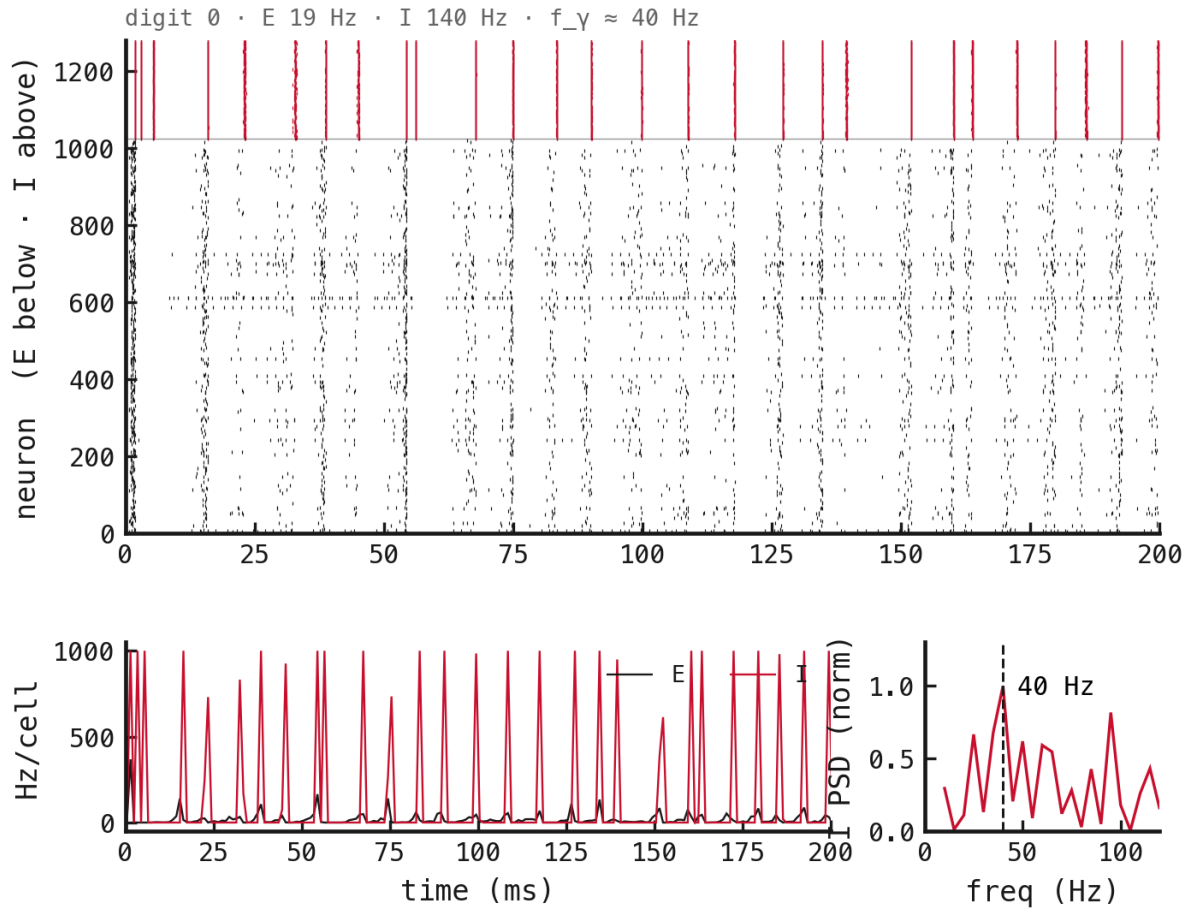


Figure 13: PING, canonical (no spike budget, all MNIST).

## A.2 Spike-budget sweep

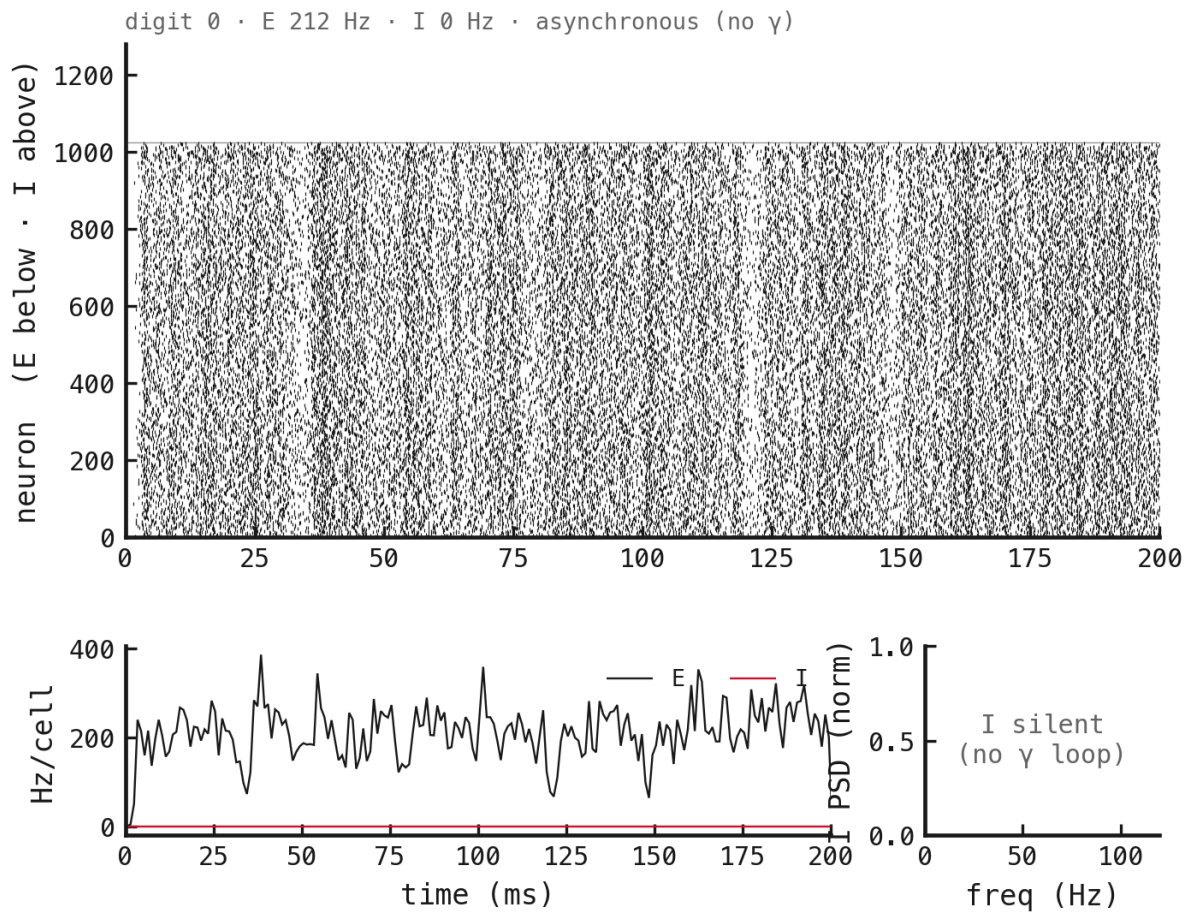


Figure 14: COBA, spike budget = off.

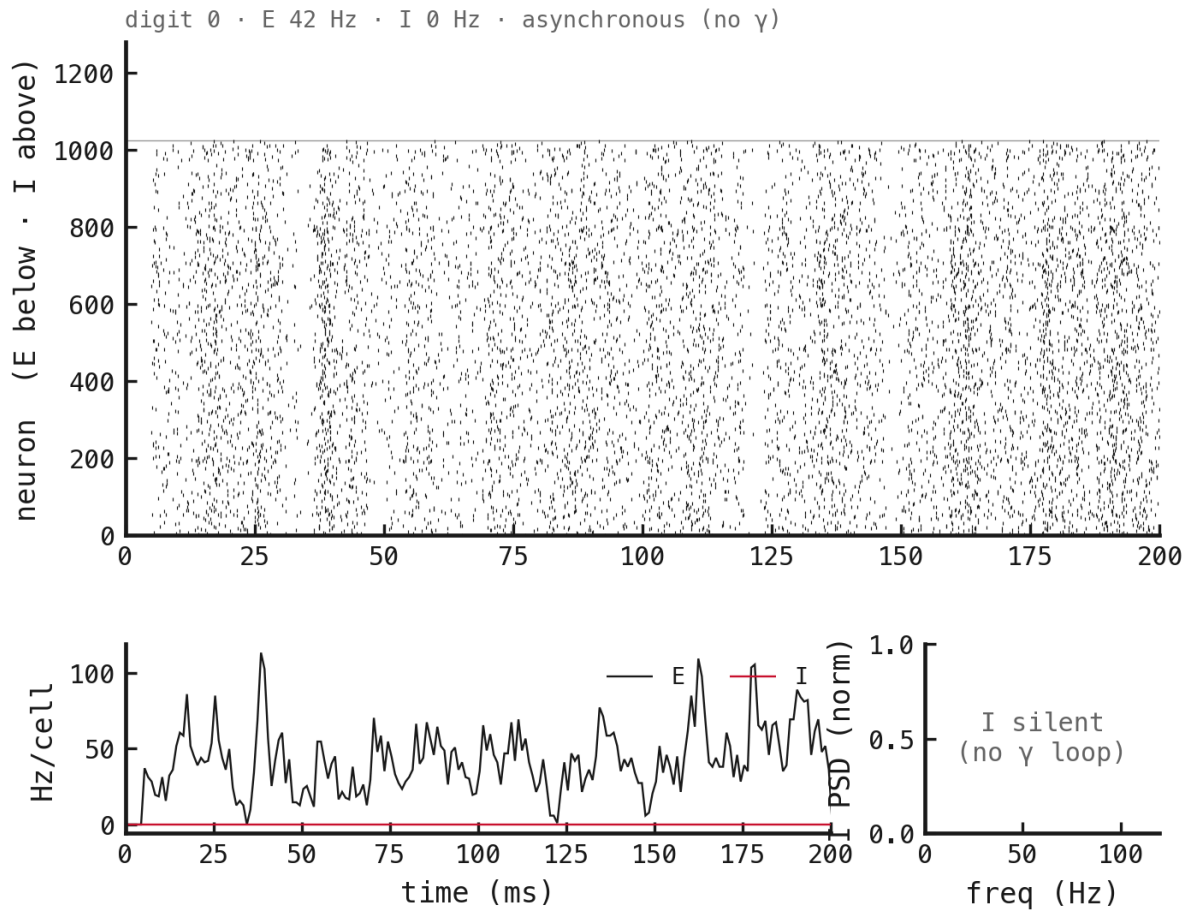


Figure 15: COBA, spike budget = 5.

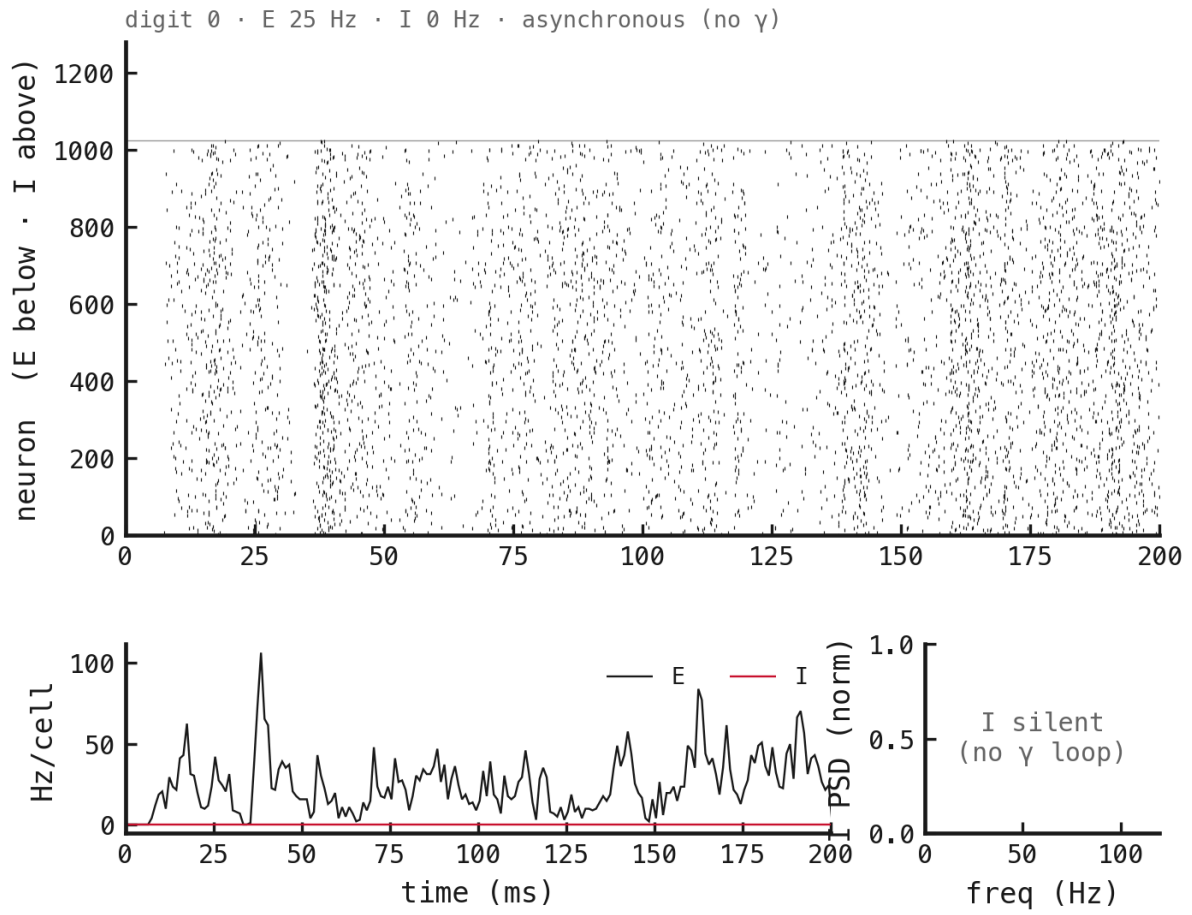


Figure 16: COBA, spike budget = 2.

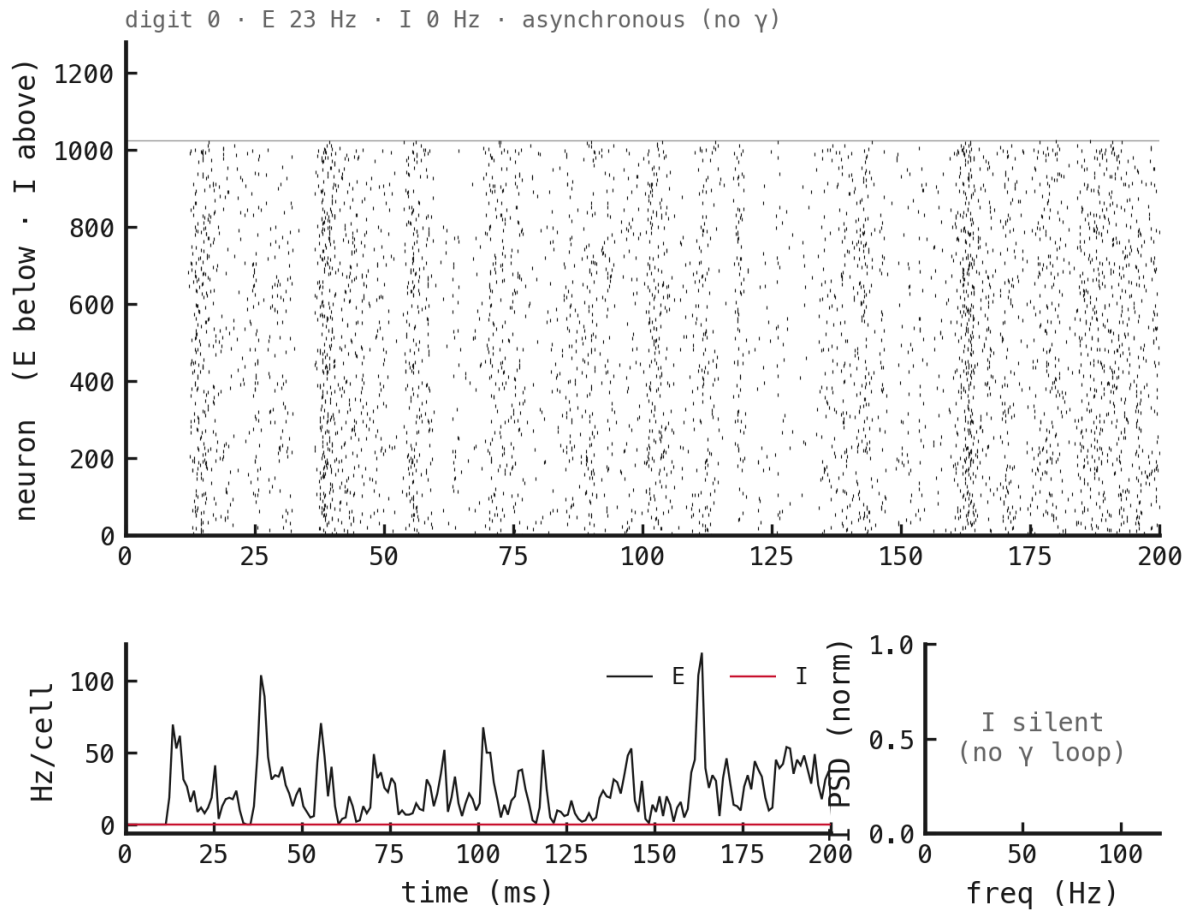


Figure 17: COBA, spike budget = 1.

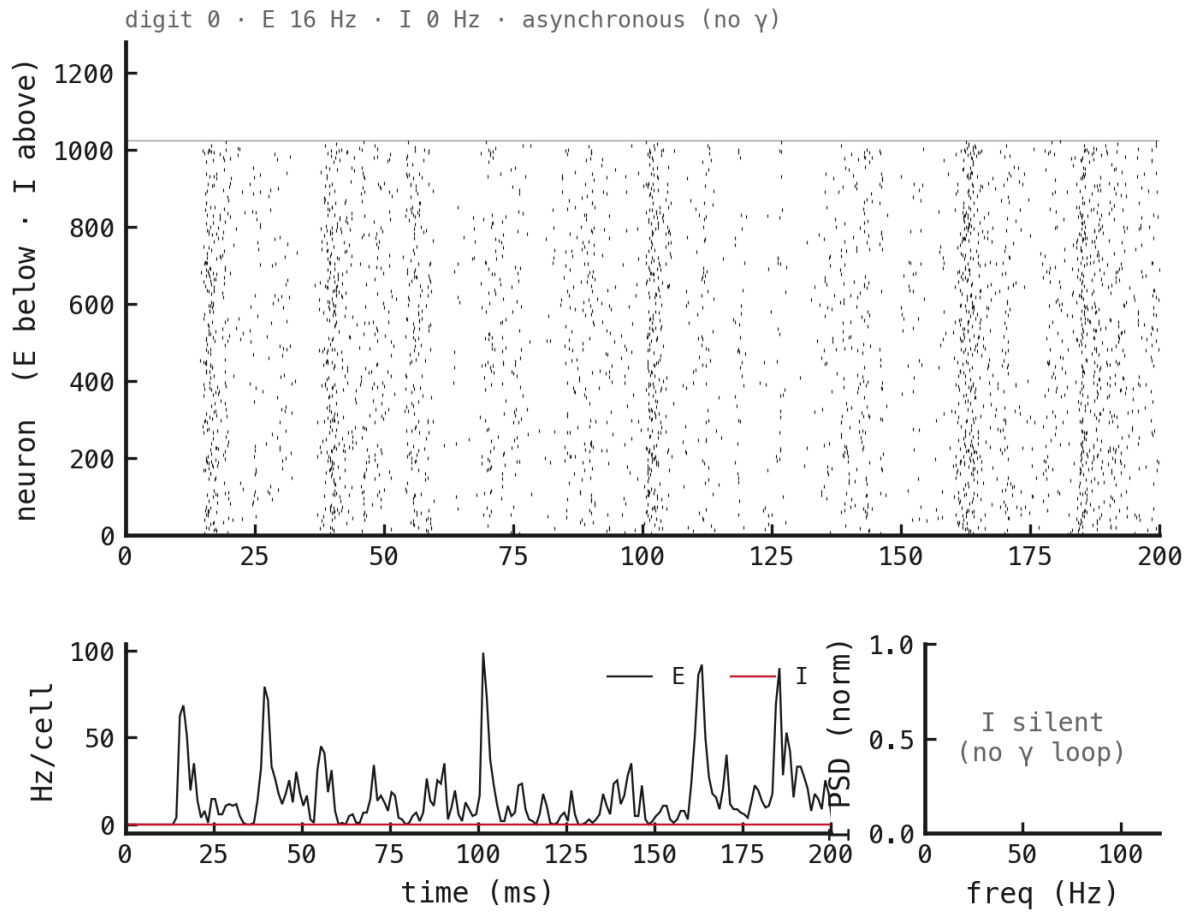


Figure 18: COBA, spike budget = 0.5.

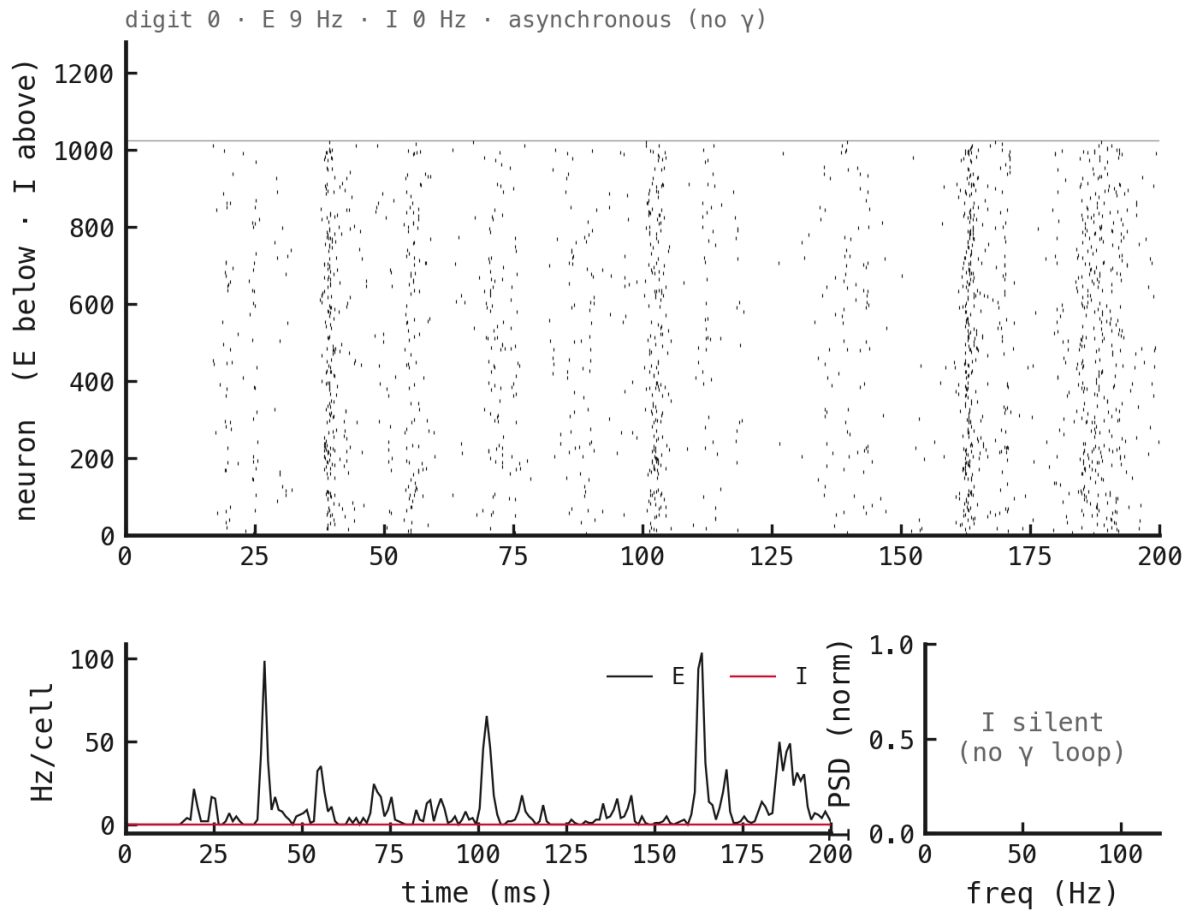


Figure 19: COBA, spike budget = 0.2.

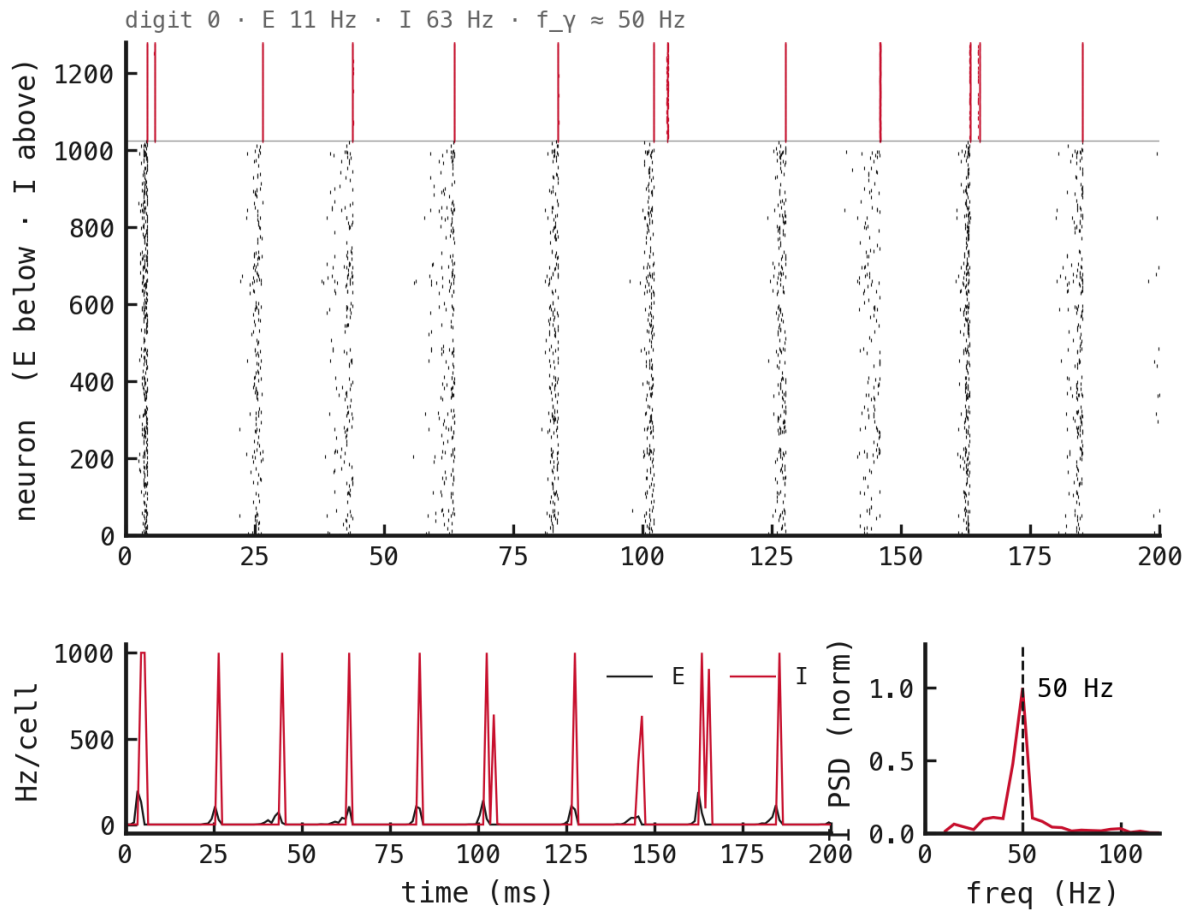


Figure 20: PING, spike budget = off.

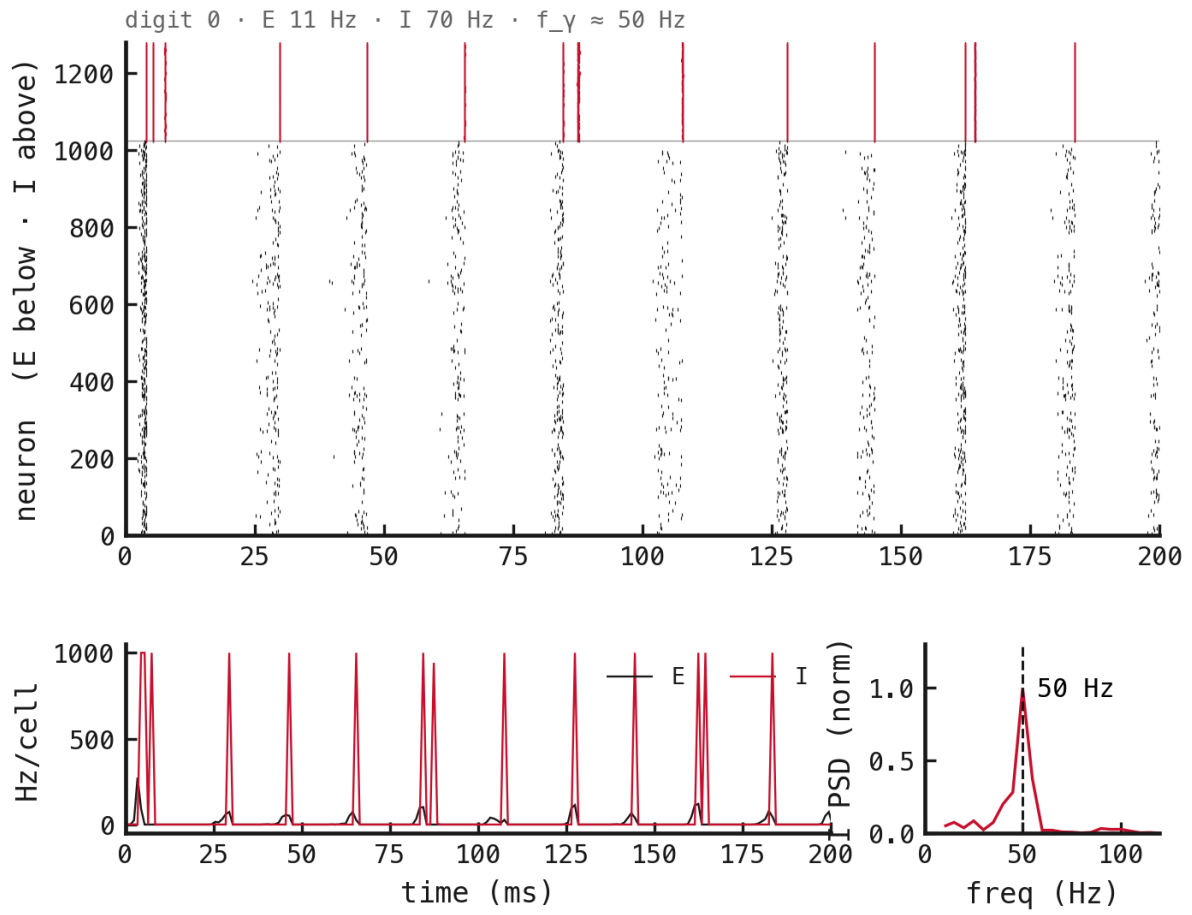


Figure 21: PING, spike budget = 5.

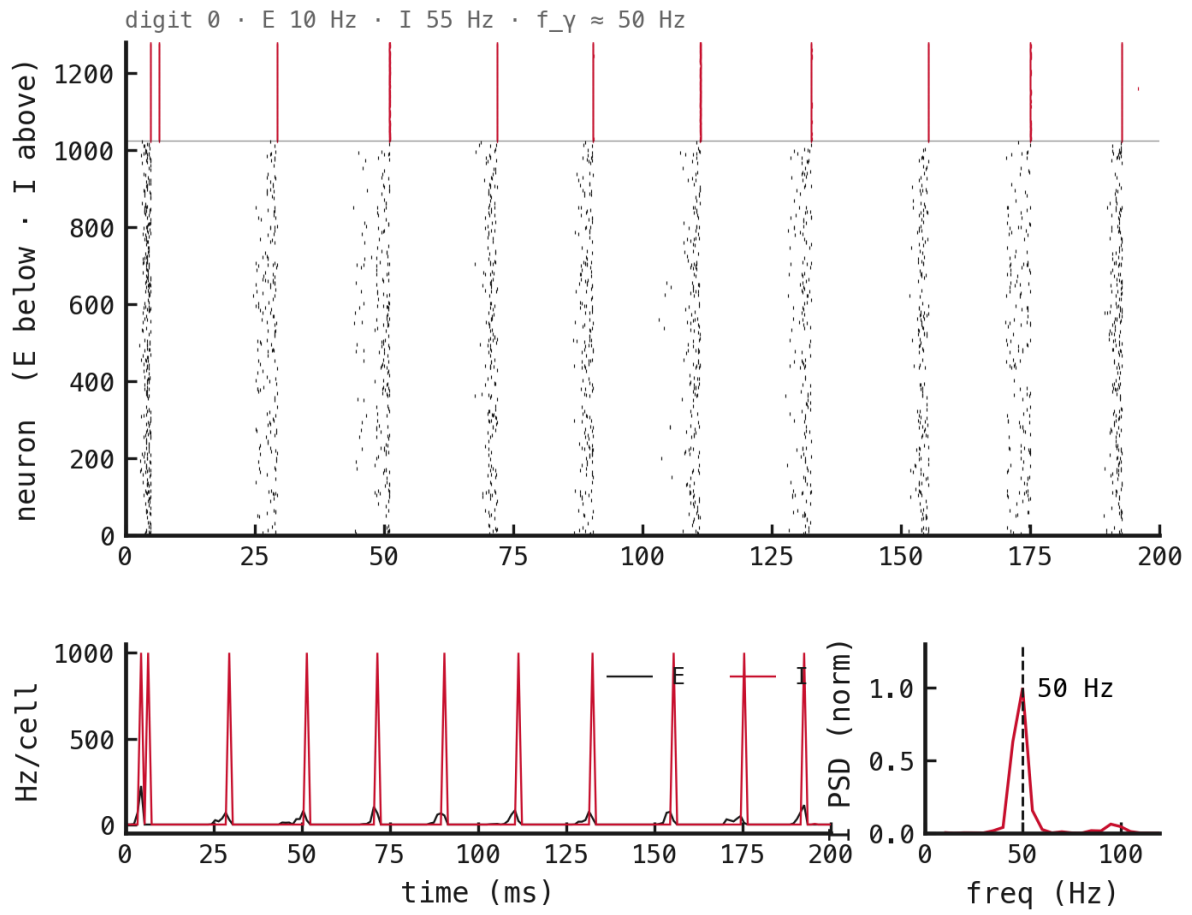


Figure 22: PING, spike budget = 2.

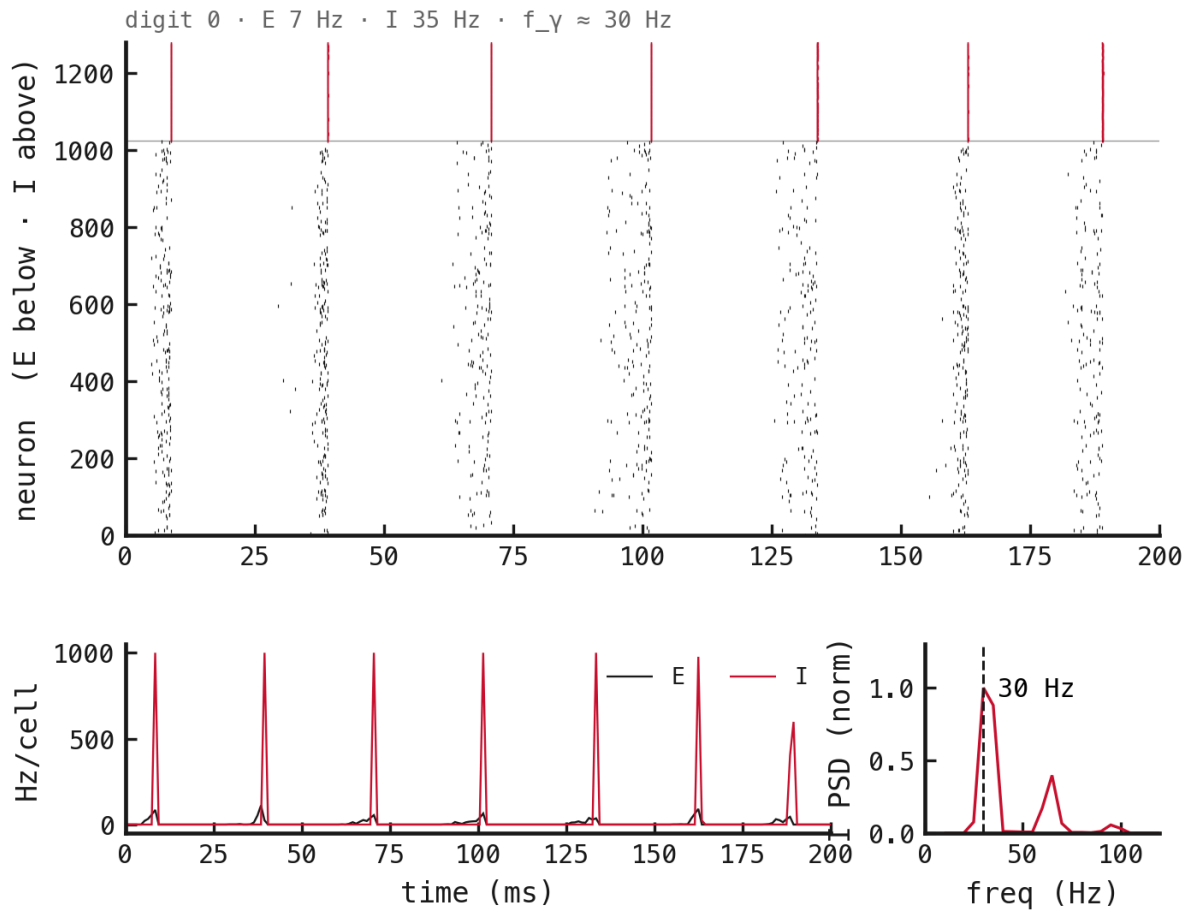


Figure 23: PING, spike budget = 1.

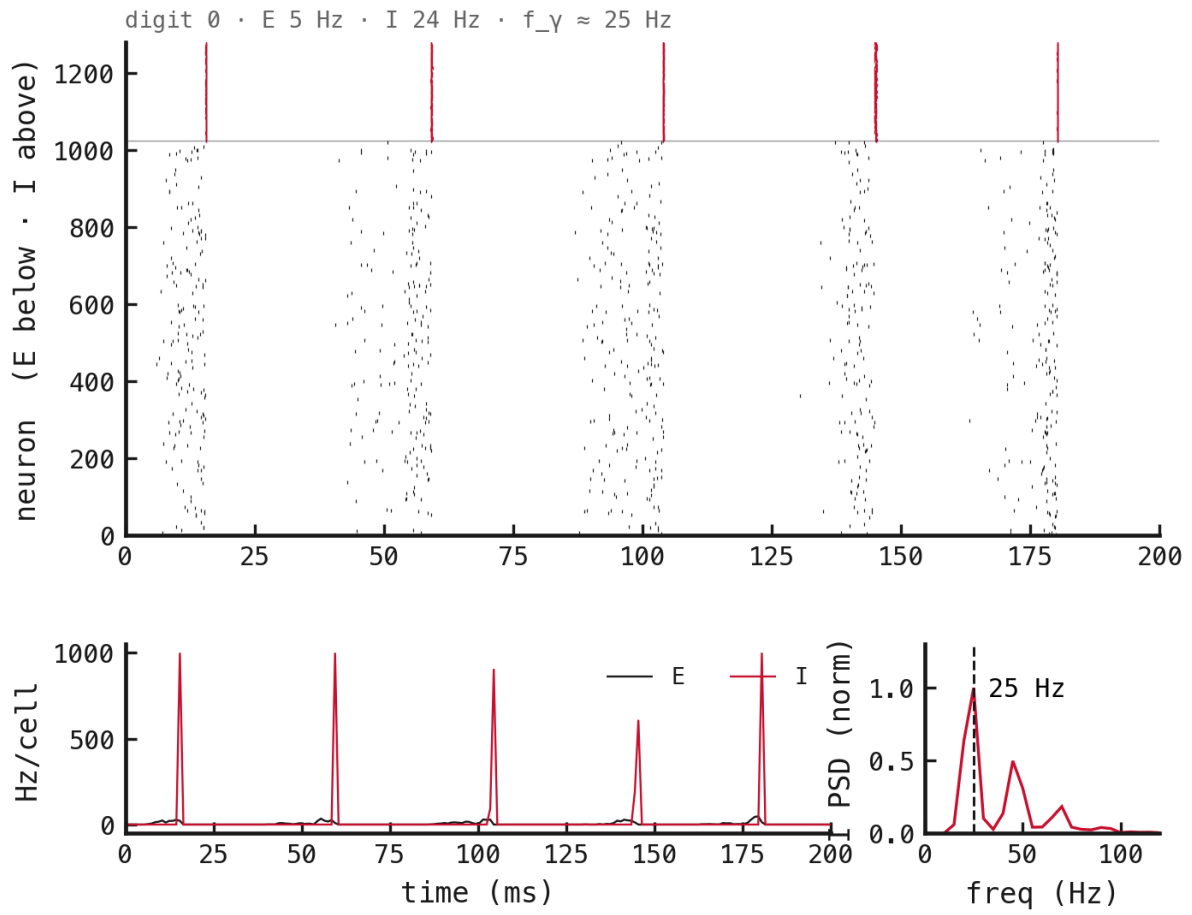


Figure 24: PING, spike budget = 0.5.

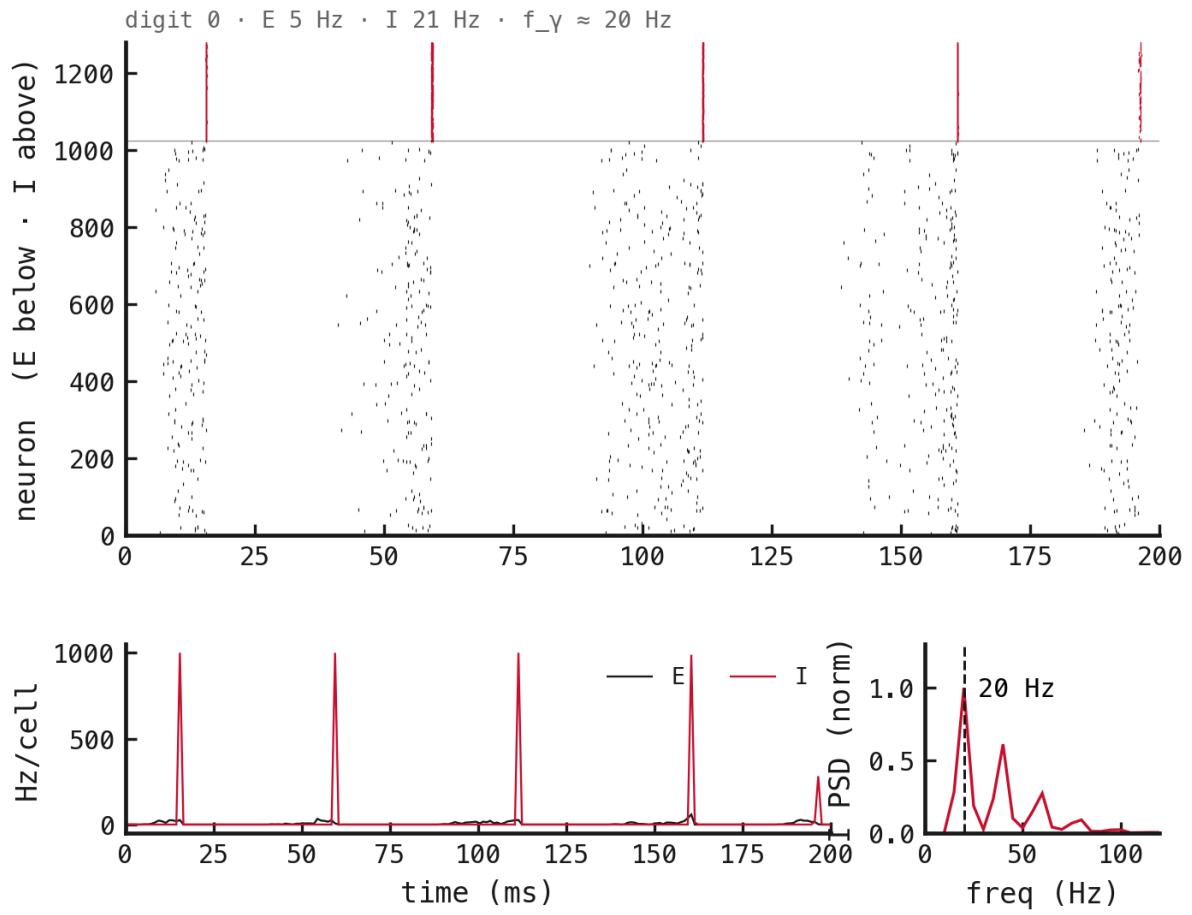


Figure 25: PING, spike budget = 0.2.

### A.3 $\tau_{\text{GABA}}$ ladder

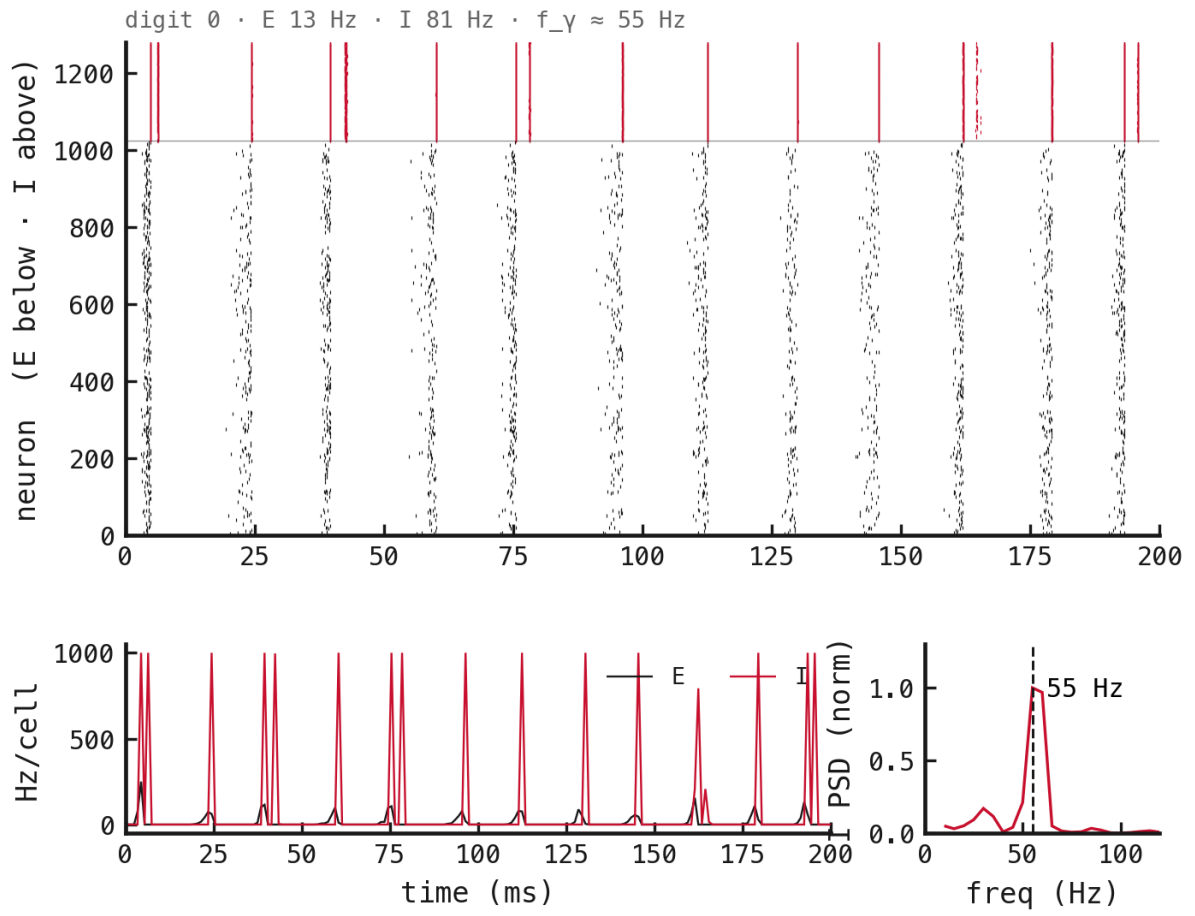


Figure 26: PING,  $\tau_{\text{GABA}} = 4.5$  ms.

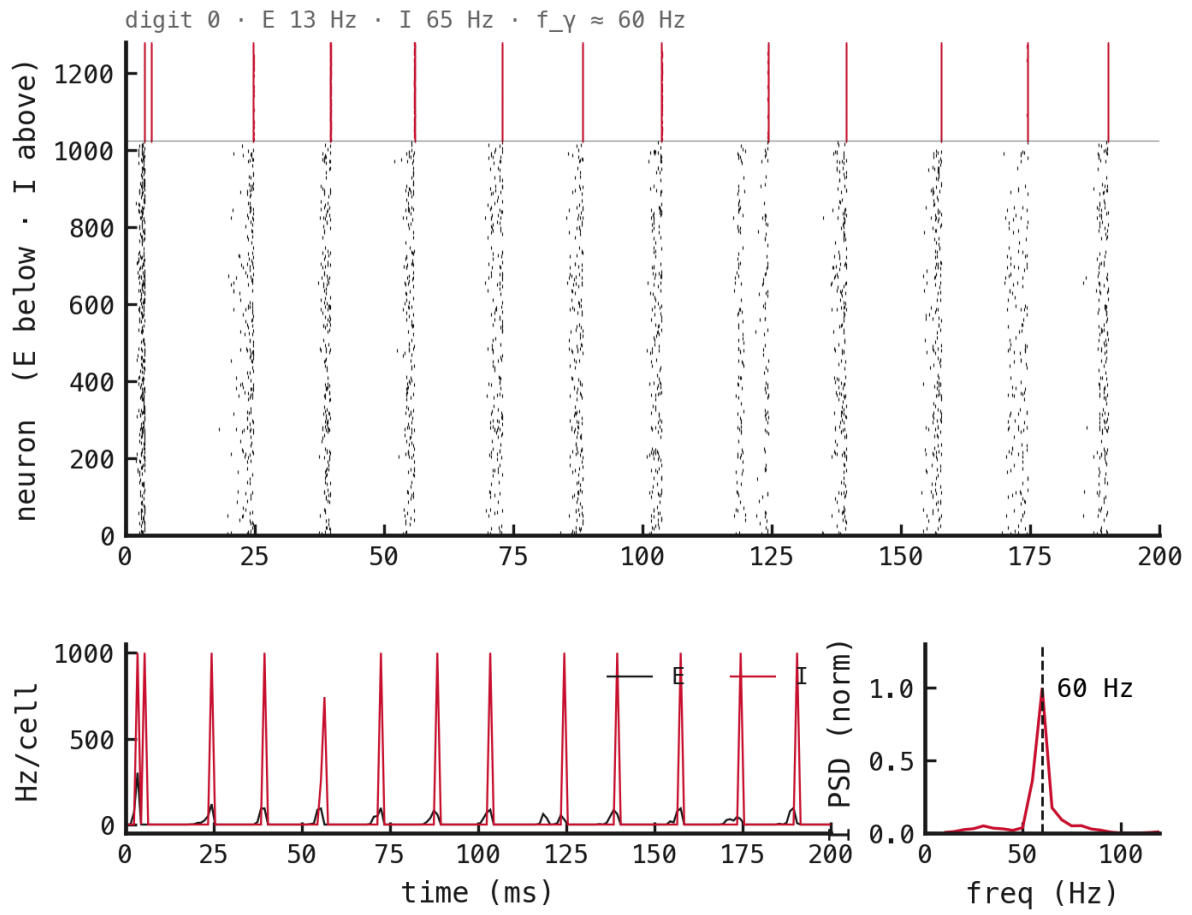


Figure 27: PING,  $\tau_{\text{GABA}} = 6$  ms.

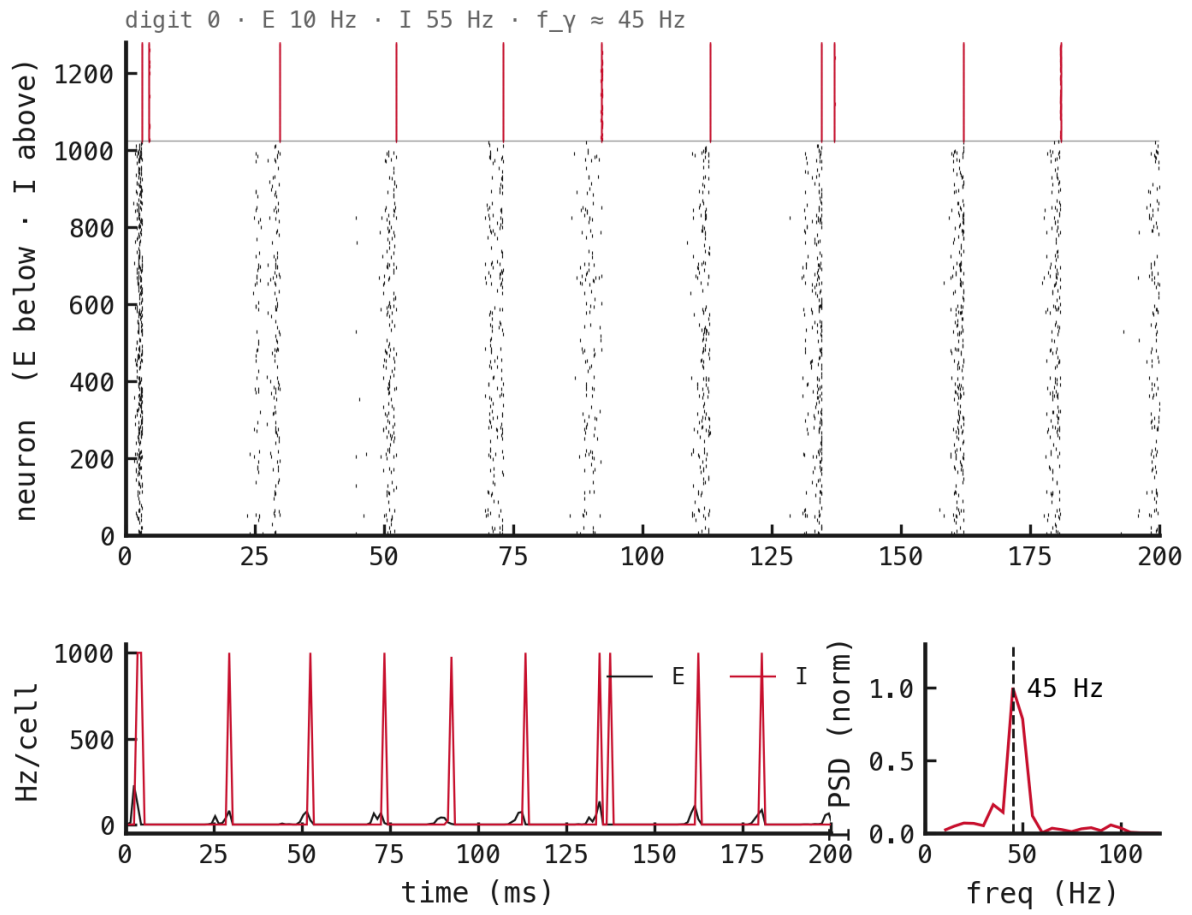


Figure 28: PING,  $\tau_{\text{GABA}} = 9$  ms.

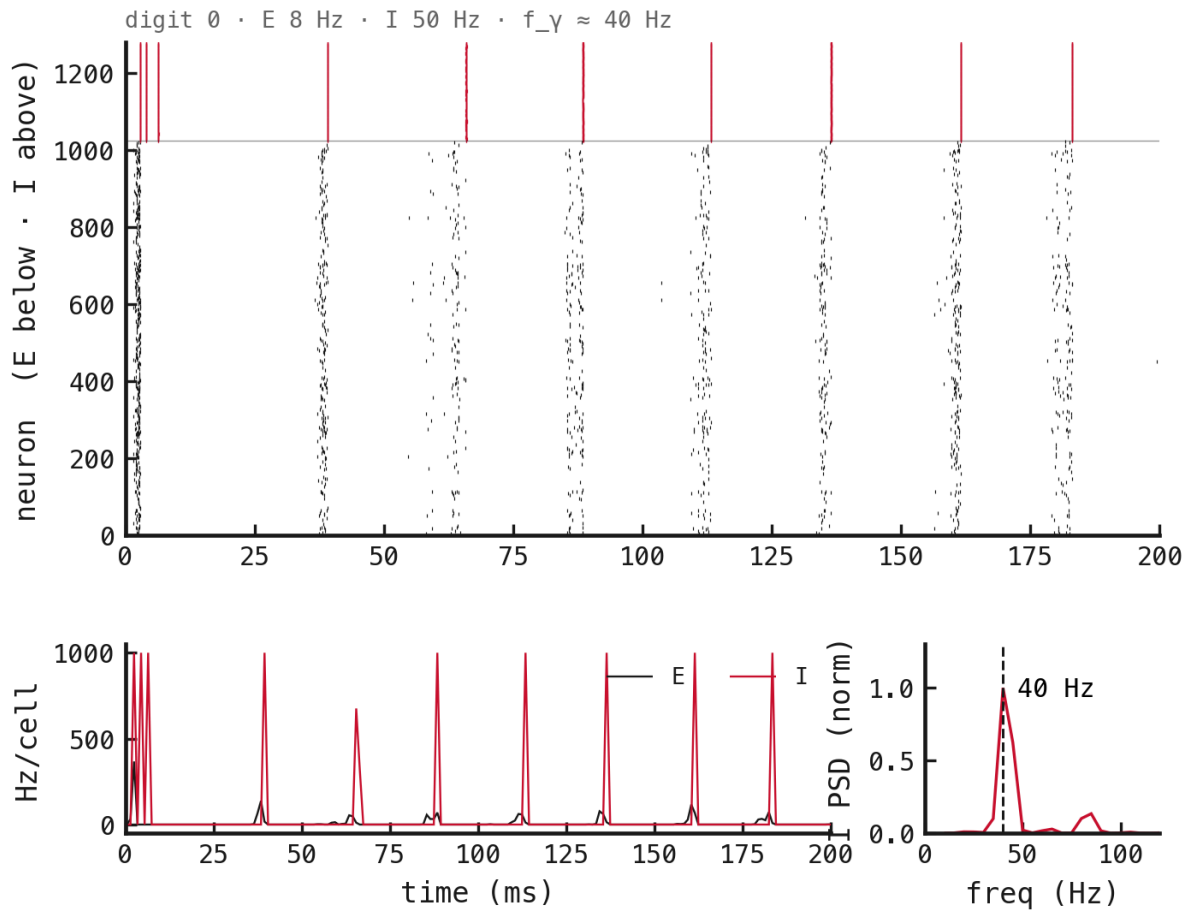


Figure 29: PING,  $\tau_{\text{GABA}} = 12$  ms.

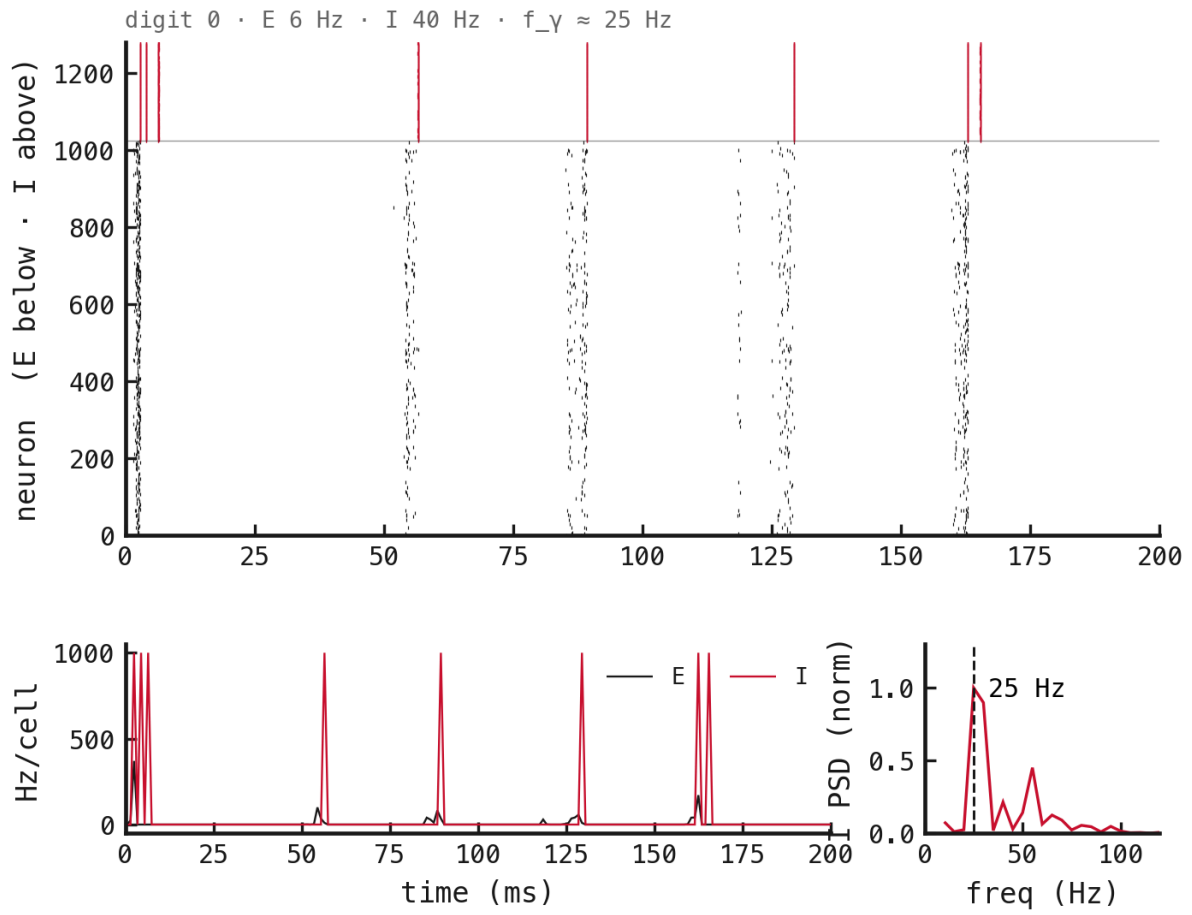


Figure 30: PING,  $\tau_{\text{GABA}} = 18$  ms.

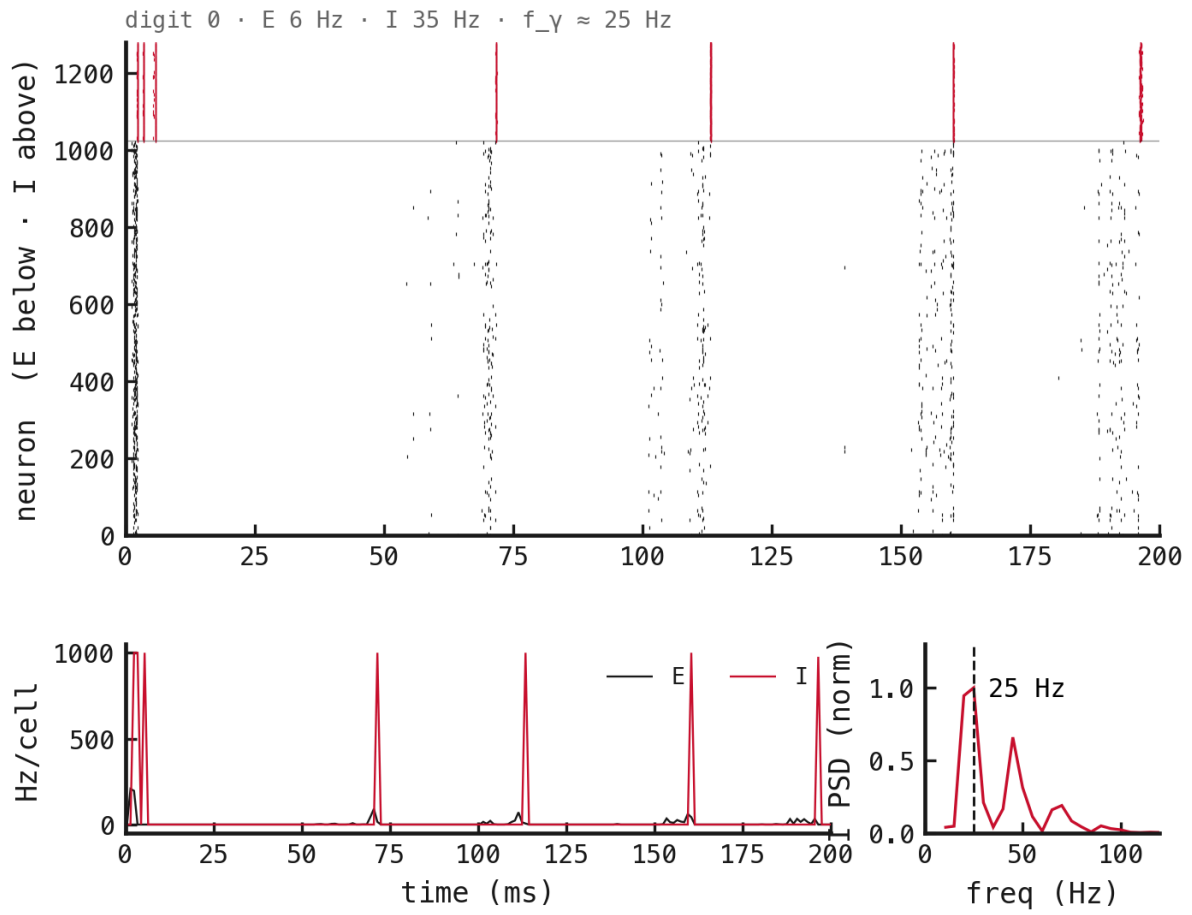


Figure 31: PING,  $\tau_{\text{GABA}} = 27$  ms.

#### A.4 $\Delta t$ sweep

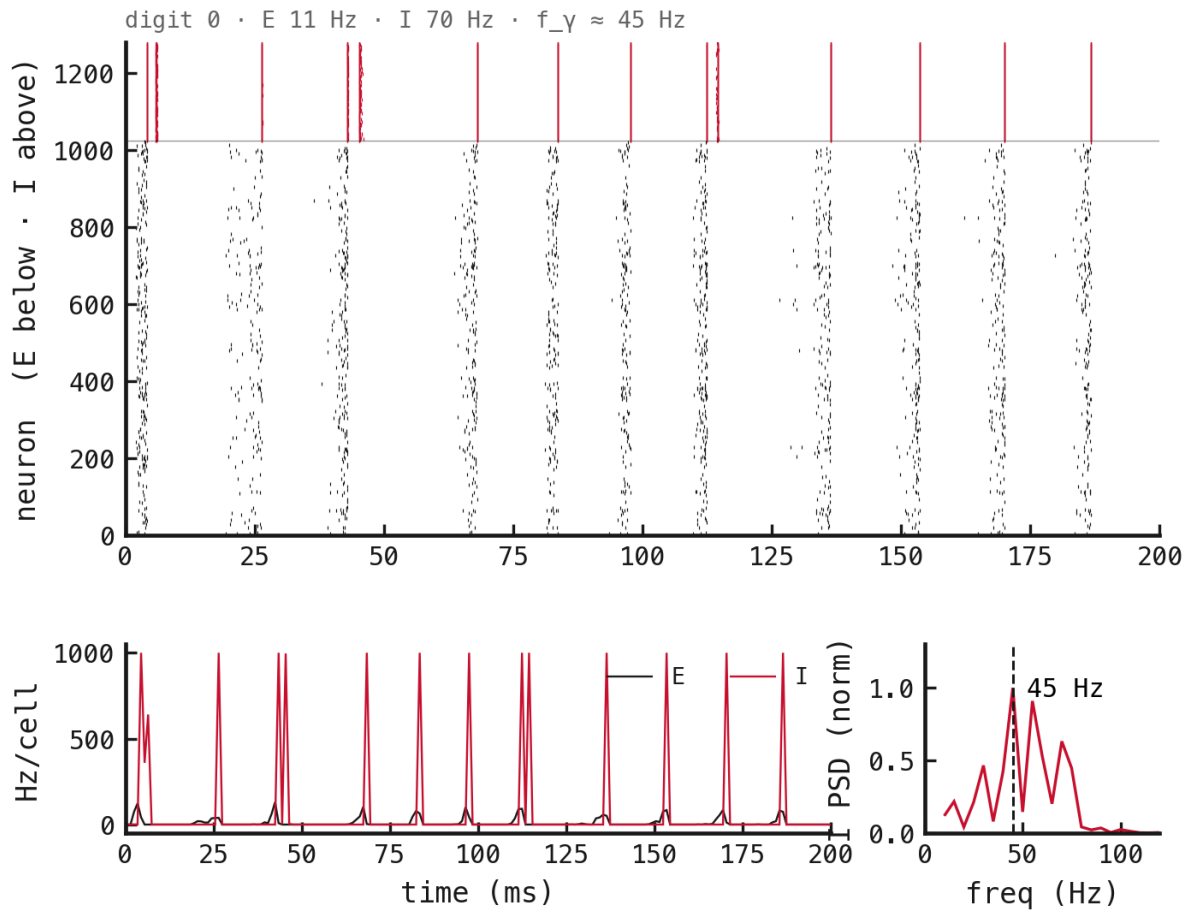


Figure 32: PING,  $\Delta t = 0.05$  ms.

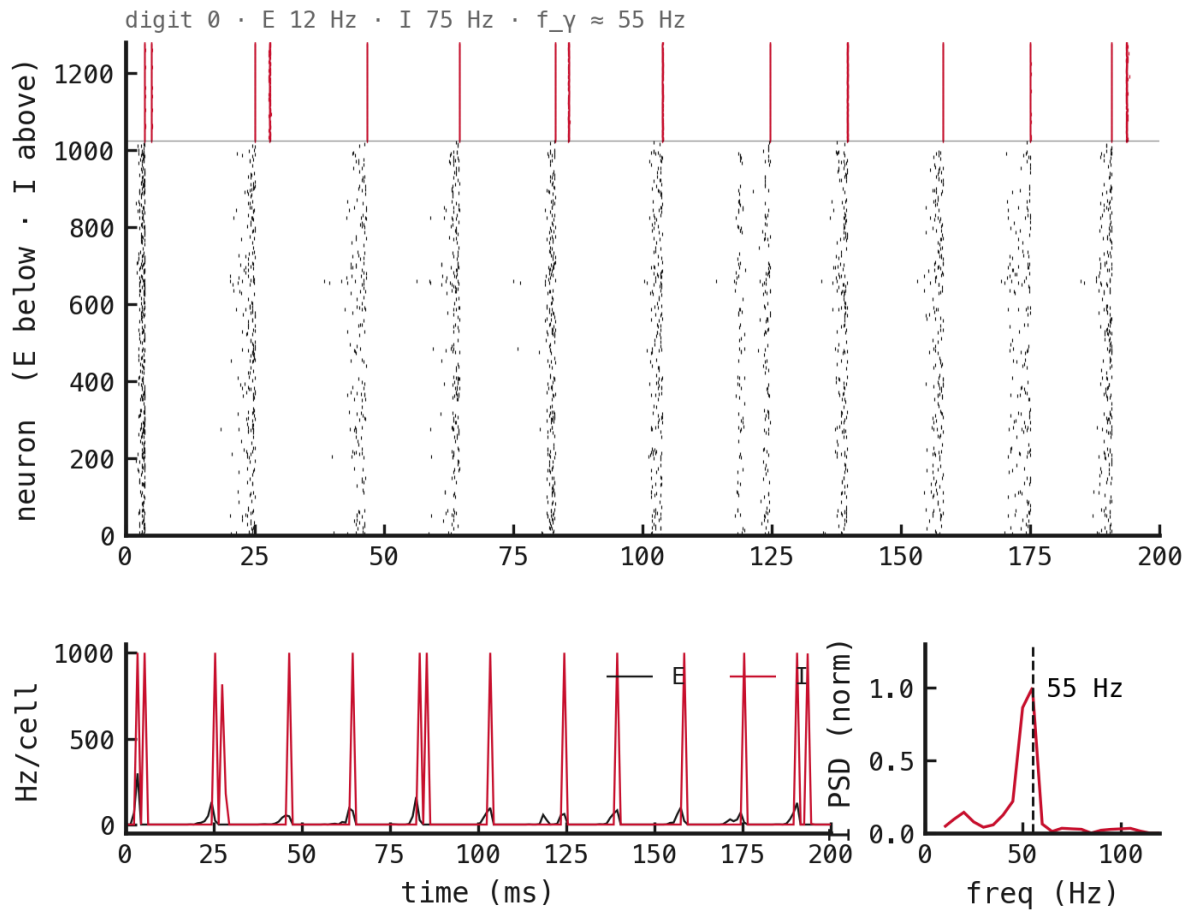


Figure 33: PING,  $\Delta t = 0.1$  ms.

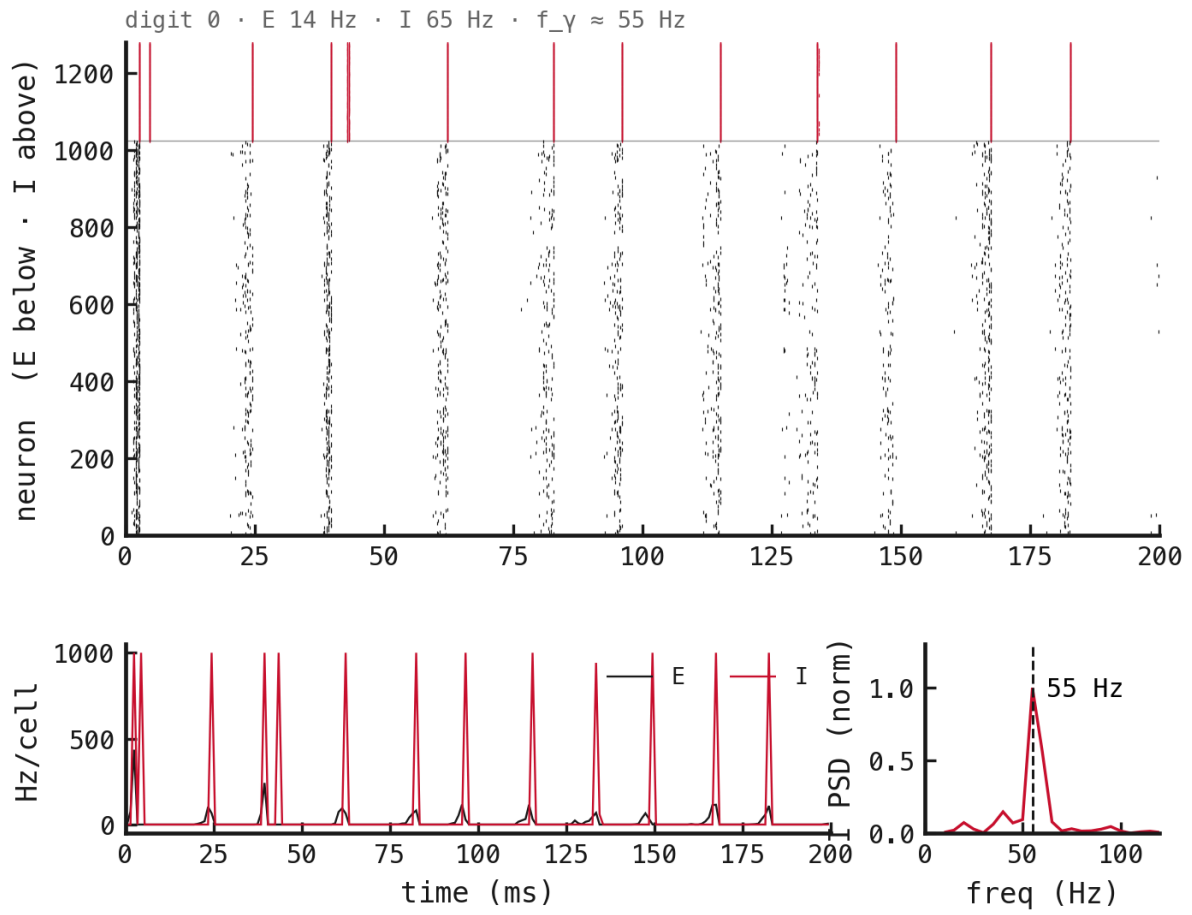


Figure 34: PING,  $\Delta t = 0.25$  ms.

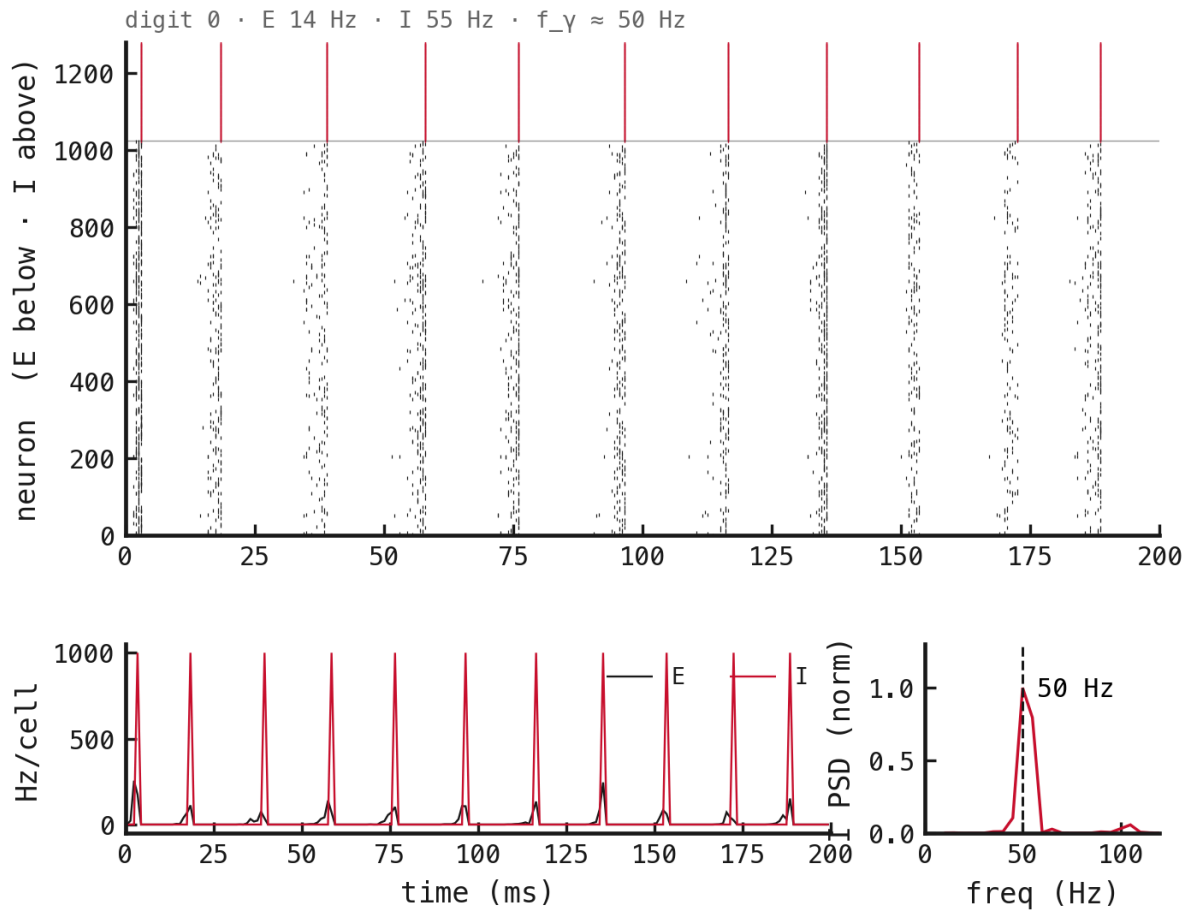


Figure 35: PING,  $\Delta t = 0.5$  ms.

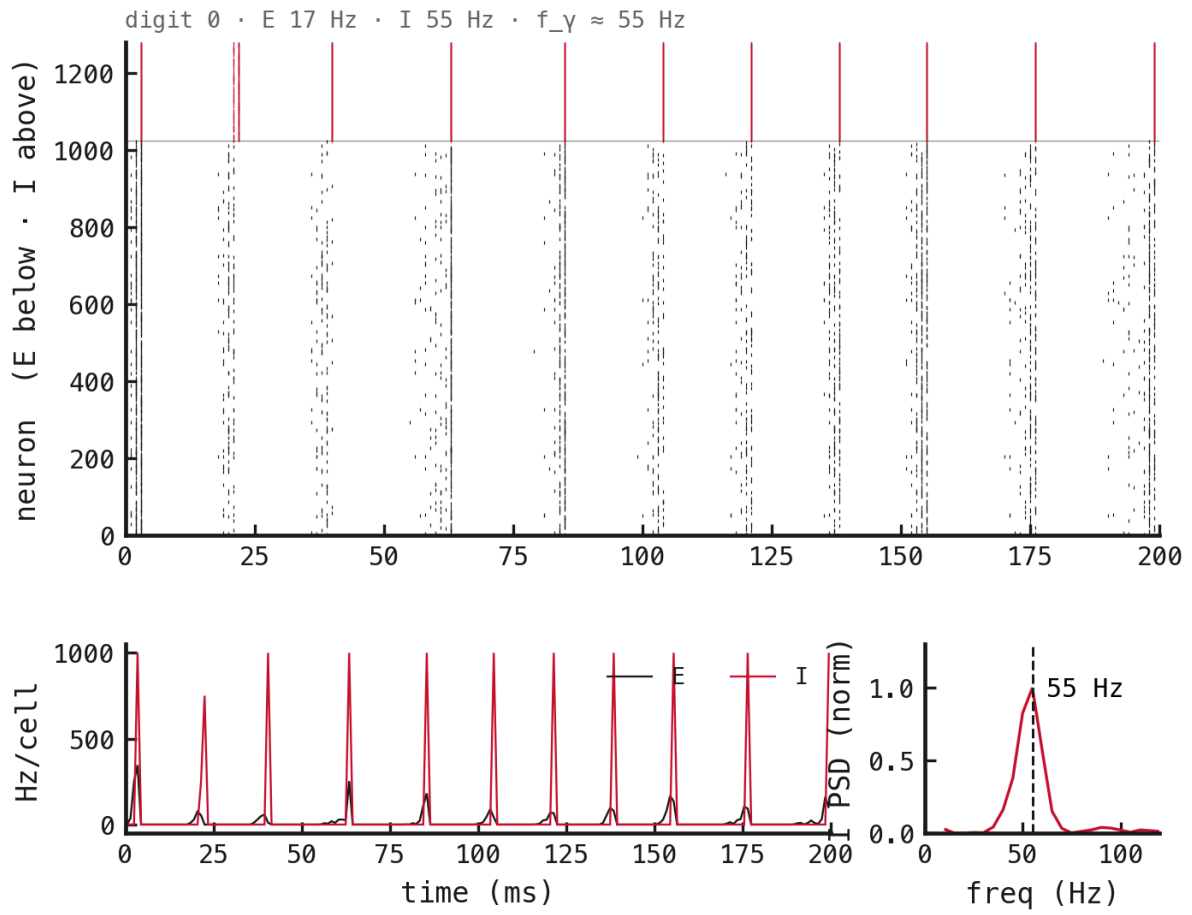


Figure 36: PING,  $\Delta t = 1$  ms.

## A.5 Init variants

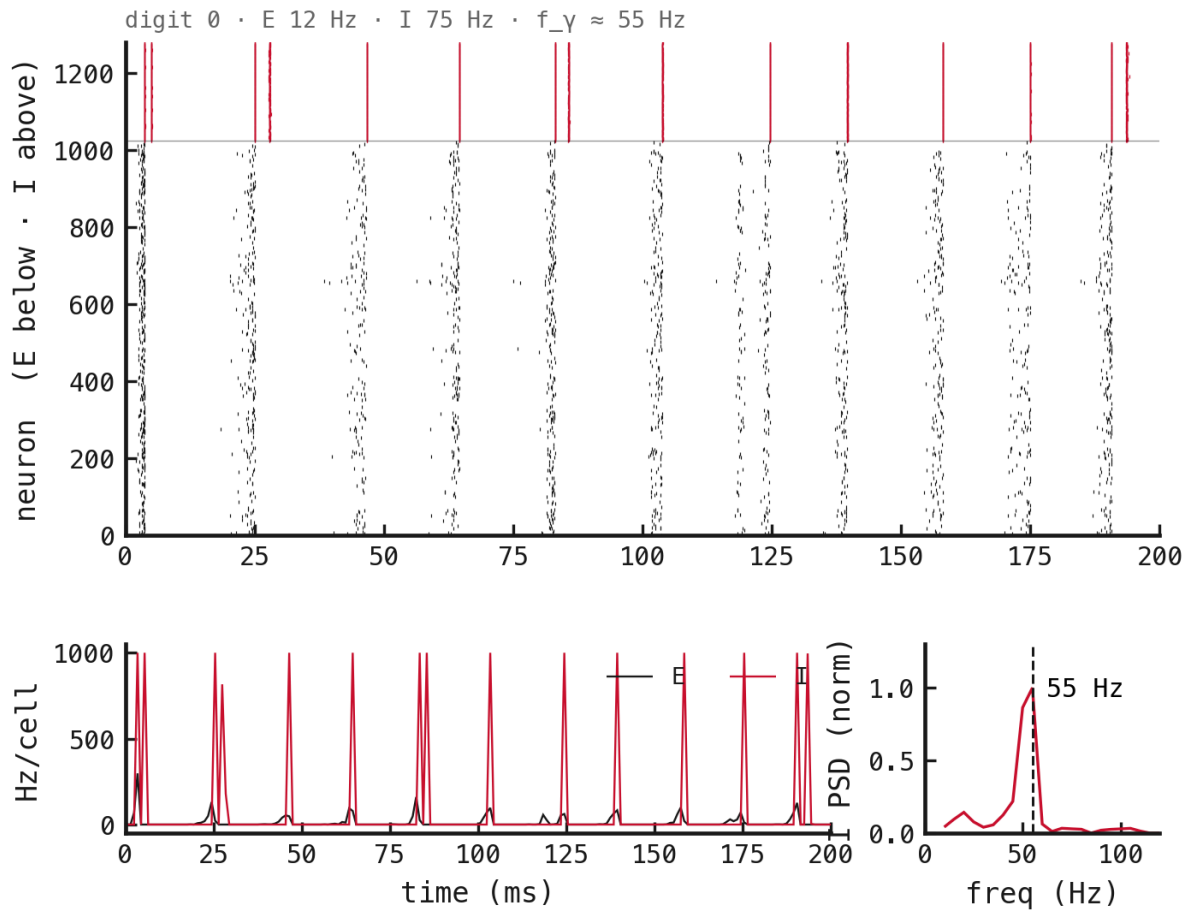


Figure 37: PING, frozen loop (control).

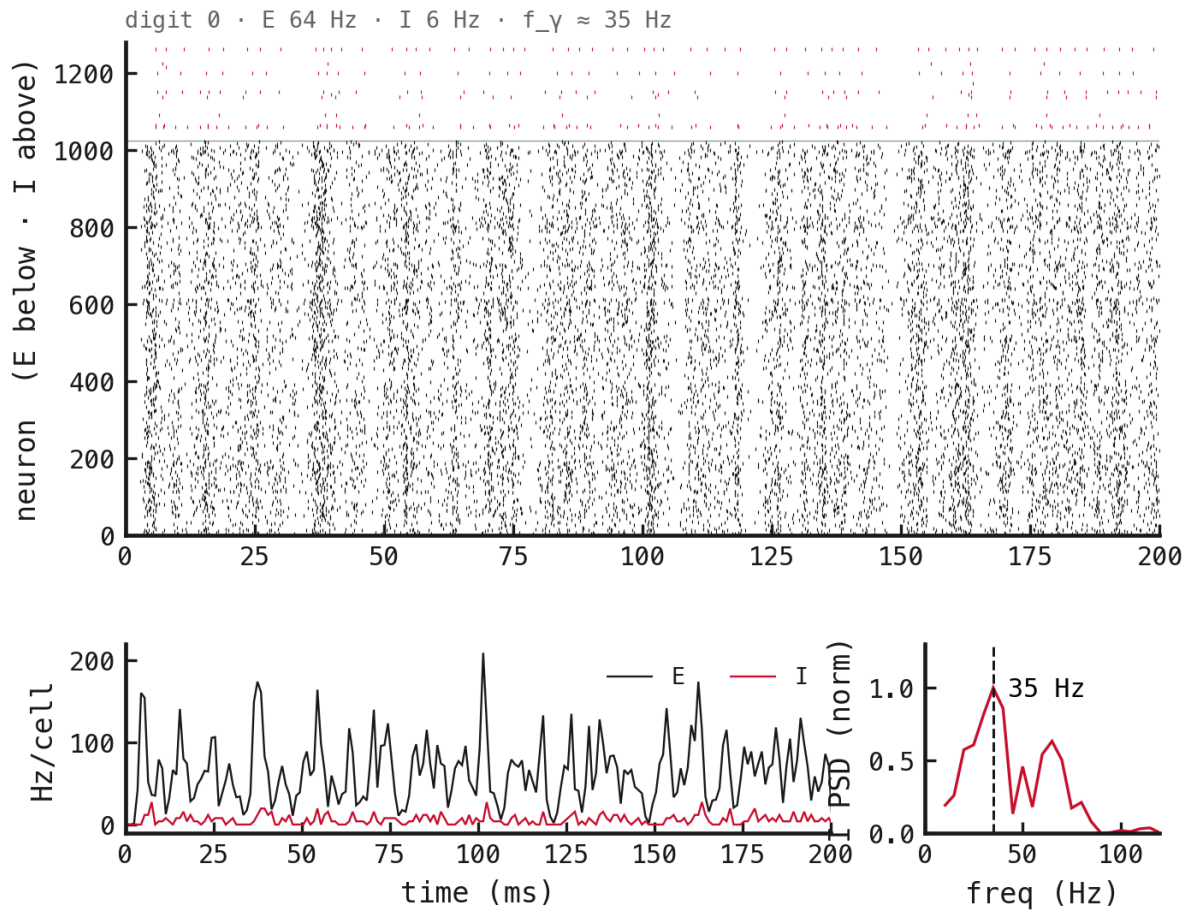


Figure 38: PING, trainable loop, PING init.

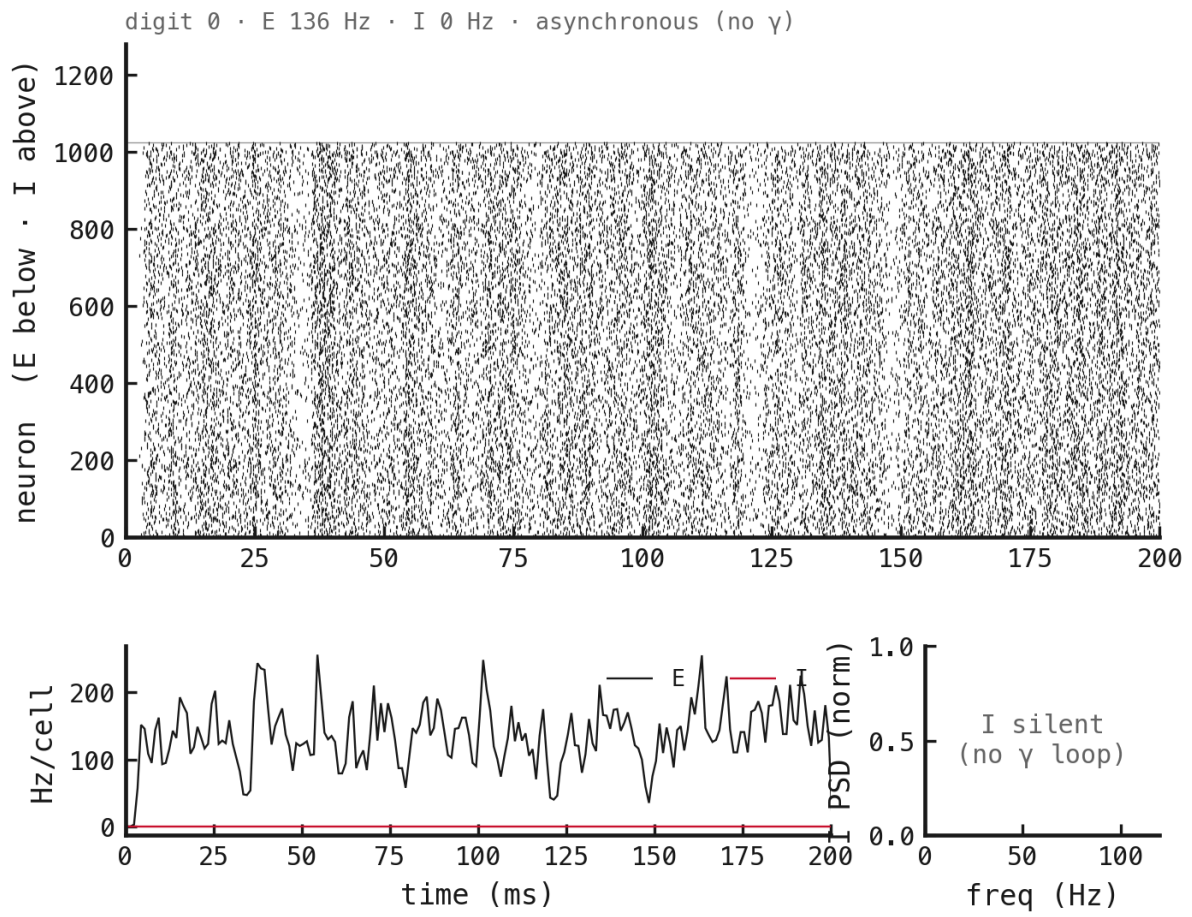


Figure 39: PING, trainable loop, zero init.

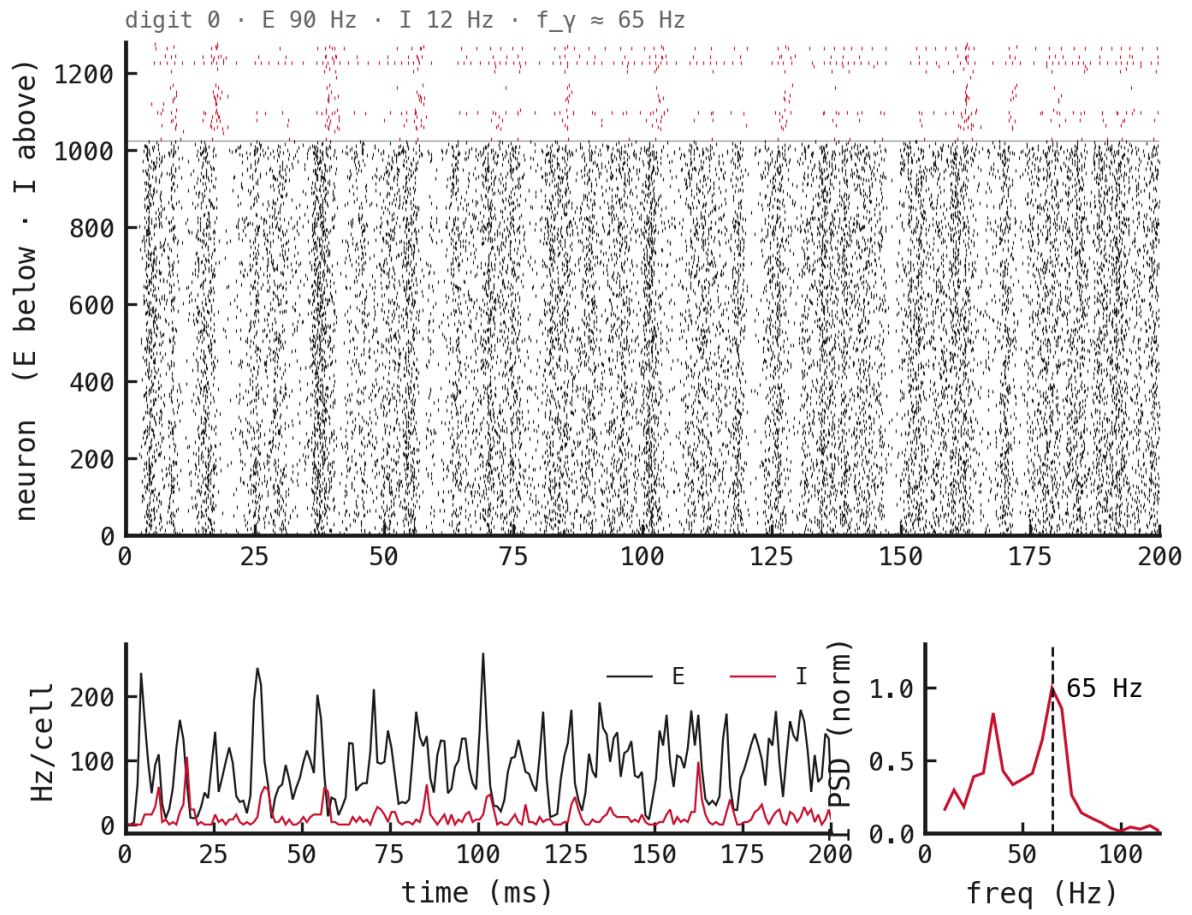


Figure 40: PING, trainable loop, small init.